

Technical Disclosure Commons

Defensive Publications Series

10 Apr 2026

NATURAL LANGUAGE QUERY TO STRUCTURED QUERY LANGUAGE (NLQ-TO-SQL) WITH AUTOMATED CONTEXT GENERATION

SUDHIR PRABHURAO JANNKATTY

BHUVANJEET SINGH GANDHI

Follow this and additional works at: https://www.tdcommons.org/dpubs_series

Recommended Citation

JANNKATTY, SUDHIR PRABHURAO and GANDHI, BHUVANJEET SINGH, "NATURAL LANGUAGE QUERY TO STRUCTURED QUERY LANGUAGE (NLQ-TO-SQL) WITH AUTOMATED CONTEXT GENERATION", Technical Disclosure Commons, (April 10, 2026)

https://www.tdcommons.org/dpubs_series/9771



This work is licensed under a [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/).

This Article is brought to you for free and open access by Technical Disclosure Commons. It has been accepted for inclusion in Defensive Publications Series by an authorized administrator of Technical Disclosure Commons.

**“NATURAL LANGUAGE QUERY TO STRUCTURED QUERY
LANGUAGE (NLQ-TO-SQL) WITH AUTOMATED CONTEXT
GENERATION”**

VISA

INVENTORS:

SUDHIR PRABHURAO JANNKATTY

BHUVANJEET SINGH GANDHI

TECHNICAL FIELD

[0001] The present subject matter is generally related to the field of natural language processing and database query systems. More specifically, the invention pertains to a method and a system for converting Natural Language Query to Structured Query Language (NQL-to-SQL) with automated content generation.

BACKGROUND

[0002] In general, data analysts and business users frequently require access to information stored in enterprise databases. Traditionally, obtaining such information requires writing Structured Query Language (SQL) queries, which demands specialized technical knowledge. Manual SQL writing creates a significant bottleneck, where data analysts spend hours to days authorizing SQL queries for business users, resulting in increased cost, delayed insights, and reduced organizational agility. Existing text-to-SQL systems suffer from several critical limitations which prevent their effective deployment in enterprise environments.

Search Scalability Issues:

[0003] Traditional systems use $O(n)$ linear search across metadata, causing performance degradation at enterprise scale with an increasing number of tables and columns (for example, 10,000 or more columns). As schema sizes grow, search operations take seconds and become unusable for real-time applications. Vector-based Retrieval Augmented Generation (RAG) approaches send large volumes of irrelevant columns, for example, 50-100 irrelevant columns, to a Large Language Model (LLM). The inclusion of excessive and irrelevant columns increases token costs to \$0.008-\$0.015 per query, and confusing the LLM with noise, which degrades query generation quality (for example, only 85% accuracy).

Metadata Bottleneck:

[0004] Enriching raw database schemas with business context, including descriptions, relationships, and aggregation rules, requires 5-7 weeks of manual work per table by data analysts and business Subject Matter Experts (SMEs). This prevents rapid onboarding of new data sources and blocks self-service analytics capabilities.

Pattern Bottleneck:

[0005] Existing systems do not reuse previously validated SQL query templates and instead generate queries anew for each request, leading to inefficiencies, inconsistent query structures,

and repeated formulation errors. Further, existing systems achieve low accuracy, approximately 76-82% accuracy due to schema hallucinations, where the LLM generates references to non-existent tables or columns, and semantic misunderstandings of user intent. Moreover, existing systems do not leverage validated SQL templates, requiring the LLM to generate queries from scratch each time, leading to inconsistent results and repeated errors, and debugging/fixing errors in existing systems require manual intervention, creating additional burden on technical staff.

[0006] Accordingly, there exists a need for an improved system which addresses the limitations through automated metadata generation, constant-time search operations, intelligent caching, self-healing capabilities, and continuous learning from user feedback.

SUMMARY

[0007] The present disclosure provides an Adaptive Translation and Learning Analytics System (ATLAS) that transforms natural language questions into accurate SQL queries and intelligent insights. Unlike traditional text-to-SQL systems, ATLAS implements a four-phase pipeline architecture with continuous learning loops, self-healing capabilities, and human-in-the-loop feedback mechanisms.

[0008] The four phases of the present disclosure comprise:

- Phase 1: Automated Metadata Factory for pre-processing database schema into AI-optimized indexes;
- Phase 2: Intelligent Query Understanding for transforming natural language into validated Semantic Query Plans (SQPs);
- Phase 3: Context-Driven SQL Synthesis for converting SQPs into optimized, constraint-compliant SQL; and
- Phase 4: Guardian Loop for executing, validating, self-healing, learning, and providing intelligent insights.

[0009] The present disclosure discloses a method and a system for converting Natural Language Query to Structured Query Language (NQL-to- SQL) with automated content generation. The present disclosure suggests receiving a natural language query from a user. Thereafter, the present disclosure suggests processing the query through an automated metadata factory which maintains pre-built indexes including a hash index for O(1) constant-time keyword lookups, a vector database for semantic similarity search, a knowledge graph for

entity relationships and join paths, and a constraint store for business rules. Subsequently, the present disclosure suggests generating a Semantic Query Plan (SQP) as a database-agnostic intermediate representation and synthesizing optimized SQL using a hybrid approach combining template matching and LLM-based generation with cost-aware model routing. Further, the present disclosure suggests executing the SQL and validating results through multi-layer validation and continuously updating the query in a database. As a result, the present disclosure achieves 96.2% success rate compared to 76-82% industry baseline, 305ms end-to-end latency representing 68% improvement, 42.3% semantic cache hit rate, which is 8.5 times better than text-based caching, 0.3% schema hallucination rate representing an 86-94% reduction, and 78% self-correction rate for automatic error recovery.

BRIEF DESCRIPTION OF THE DRAWINGS

[0010] The accompanying drawings, which are incorporated in and constitute a part of this disclosure, illustrate exemplary embodiments and, together with the description, explain the disclosed principles. In the figures, the left-most digit(s) of a reference number identifies the figure in which the reference number first appears. The same numbers are used throughout the figures to reference like features and components. Some embodiments of device or system and/or methods in accordance with embodiments of the present subject matter are now described, by way of example only, and with reference to the accompanying figures, in which:

[0011] FIG. 1 illustrates an exemplary architecture of a system which may be configured to convert Natural Language Query to Structured Query Language (NQL-to- SQL) with automated content generation, in accordance with some embodiments of the present disclosure;

[0012] FIG. 2 illustrates an exemplary flow diagram of a method for converting Natural Language Query to Structured Query Language (NQL-to-SQL) with automated content generation, in accordance with some embodiments of the present disclosure;

[0013] FIG. 3 shows a high-level flow diagram illustrating the multi-phase pipeline architecture of the Natural Language Query to Structured Query Language (NQL-to-SQL) translation system, in accordance with some embodiments of the present disclosure;

[0014] FIGs. 4a and 4b illustrate an exemplary four-phase transformation summary diagram with feedback loops, in accordance with some embodiments of the present disclosure;

[0015] FIG. 5 illustrates an automated metadata generation pipeline for auto-creating metadata for new database tables, in accordance with some embodiments of the present disclosure;

[0016] FIG. 6 illustrates a comparative table evaluating three different approaches for Natural Language Query to Structured Query Language (NQL-to-SQL) conversion across capability metrics, in accordance with some embodiments of the present disclosure;

[0017] FIG. 7 illustrates a flow diagram for an automated metadata factory (Phase 1) showing the complete data flow and processing pipeline, in accordance with some embodiments of the present disclosure;

[0018] FIG. 8 illustrates a flow diagram for Query Understanding (Phase 2) showing the complete query processing pipeline, in accordance with some embodiments of the present disclosure;

[0019] FIGs. 9 and 10 illustrate a flow diagram for a constrained Structured Query Language (SQL) generation pipeline (Phase 3) showing the processing pipeline from Valid SQP input through to Valid SQL output, template matching and cost-aware routing mechanisms, in accordance with some embodiments of the present disclosure;

[0020] FIG. 11 illustrates an exemplary architecture diagram for a guardian loop (Phase 4) showing execution and validation components, in accordance with some embodiments of the present disclosure; and

[0021] FIG. 12 shows an exemplary flowchart illustrating the Natural Language Query to Structured Query Language (NQL-to-SQL) conversion process using a multi-tenant architecture, in accordance with some embodiments of the present disclosure;

[0022] The figures depict embodiments of the disclosure for purposes of illustration only. One skilled in the art will readily recognize from the following description that alternative embodiments of the structures and methods illustrated herein may be employed without departing from the principles of the disclosure described herein.

DESCRIPTION OF THE DISCLOSURE

[0023] It is to be understood that the present disclosure may assume various alternative variations and step sequences, except where expressly specified to the contrary. It is also to be understood that the specific devices and processes illustrated in the attached drawings and described in the following specification are simply exemplary and non-limiting embodiments or aspects. Hence, specific dimensions and other physical characteristics related to the embodiments or aspects disclosed herein are not to be considered as limiting.

[0024] In the present document, the word "exemplary" is used herein to mean "serving as an example, instance, or illustration." Any embodiment or implementation of the present subject matter described herein as "exemplary" is not necessarily to be construed as preferred or advantageous over other embodiments.

[0025] While the disclosure is susceptible to various modifications and alternative forms, specific embodiment thereof has been shown by way of example in the drawings and will be described in detail below. It should be understood, however that it is not intended to limit the disclosure to the particular forms disclosed, but on the contrary, the disclosure is to cover all modifications, equivalents, and alternative falling within the spirit and the scope of the disclosure.

[0026] The terms “comprises”, “comprising”, or any other variations thereof, are intended to cover a non-exclusive inclusion, such that a setup, device, or method that comprises a list of components or steps does not include only those components or steps but may include other components or steps not expressly listed or inherent to such setup or device or method. In other words, one or more elements in a device or system or apparatus preceded by “comprises... a” does not, without more constraints, preclude the existence of other elements or additional elements in the device or system or apparatus.

[0027] The terms "an embodiment", "embodiment", "embodiments", "the embodiment", "the embodiments", "one or more embodiments", "some embodiments", and "one embodiment" mean "one or more (but not all) embodiments of the invention(s)" unless expressly specified otherwise.

[0028] The terms "including", "comprising", “having” and variations thereof mean "including but not limited to" unless expressly specified otherwise.

[0029] As used herein, the terms “communication” and “communicate” may refer to the reception, receipt, transmission, transfer, provision, and/or the like of information (e.g., data, signals, messages, instructions, commands, and/or the like). For one unit (e.g., a device, a system, a component of a device or system, combinations thereof, and/or the like) to be in communication with another unit means that the one unit can receive information directly or indirectly from and/or transmit information to the other unit. This may refer to a direct or indirect connection (e.g., a direct communication connection, an indirect communication connection, and/or the like) that is wired and/or wireless in nature. Additionally, two units may be in communication with each other even though the information transmitted may be modified, processed, relayed, and/or routed between the first and second unit.

[0030] As used herein, the term "application" may refer to a computer code or other data stored on a computer-readable medium (e.g., memory element or secure element) that may be executable by a processor to complete a task.

[0031] As used herein, the term “computing device” may refer to one or more electronic devices that are configured to communicate with directly or indirectly or over one or more networks. A computing device may be a mobile or portable computing device, a desktop computer, a server, and/or the like. Furthermore, the term “computer” may refer to any computing device that includes the necessary components to receive, process, and output data, and normally includes a display, a processor, a memory, an input device, and a network interface. A “computing system” may include one or more computing devices or computers.

[0032] As used herein, the term "processor" may refer to any suitable data computation device or devices. A processor may comprise one or more microprocessors working together to accomplish a desired function.

[0033] As used herein, the term "memory" may be any suitable device or devices that can store electronic data. A suitable memory may comprise a non-transitory computer readable medium that stores instructions that can be executed by a processor to implement a desired method. Examples of memories may comprise one or more memory chips, disk drives, etc. Such memories may operate using any suitable electrical, optical, and/or magnetic mode of operation.

[0034] It will be apparent that systems and/or methods described herein can be implemented in different forms of hardware, software, or a combination of hardware and software. The actual specialized control hardware or software code used to implement these systems and/or methods is not limiting of the implementations. Thus, the operation and behavior of the systems and/or methods are described herein without reference to specific software code, it being understood that software and hardware can be designed to implement the systems and/or methods based on the description herein.

[0035] The present disclosure provides an Adaptive Translation and Learning Analytics System (ATLAS) that transforms natural language questions into accurate SQL queries and intelligent insights. Unlike traditional text-to-SQL systems, ATLAS implements a four-phase pipeline architecture with continuous learning loops, self-healing capabilities, and human-in-the-loop feedback mechanisms.

[0036] The present disclosure encompasses the following main concepts which are detailed in further sections of the disclosure.

[0037] Four-Phase Pipeline Architecture: A systematic approach dividing natural language to SQL conversion into distinct phases: Automated Metadata Factory (Phase 1), Intelligent Query Understanding (Phase 2), Context-Driven SQL Synthesis (Phase 3), and Guardian Loop (Phase 4), with continuous feedback loops enabling self-improvement.

[0038] O(1) Hash-Based Inverted Index: A novel data structure providing constant-time keyword-to-column mapping regardless of schema size, replacing O(n) linear search with 0.000-second lookups at any scale from 1,000 to 1,000,000 columns.

[0039] Semantic Query Plan (SQP) Intermediate Representation: A database-agnostic semantic layer that separates query understanding from SQL generation, enabling caching, validation, and portability across different database systems.

[0040] EWSRP Algorithm (Entropy-Weighted Semantic Relevance Pruning): A context pruning algorithm applying Shannon entropy weighting to achieve 82% token reduction while maintaining 99.7% relevance accuracy.

[0041] MOBOR Algorithm (Multi-Objective Bayesian Optimization Routing): A cost-aware model routing algorithm using Bayesian surrogate models, Pareto frontier computation, and Thompson Sampling to optimize the cost-quality tradeoff in LLM selection.

[0042] Temporal Knowledge Graph (TKG): A versioned knowledge graph capturing entity relationships, valid join paths, and trust scores with schema change tracking and drift detection.

[0043] Multi-Level Semantic Cache: A three-tier caching strategy (exact fingerprint, parametric template, semantic similarity) achieving 42.3% hit rate through intent-based rather than text-based matching.

[0044] Self-Healing Guardian Loop: An execution and validation framework with 78% automatic error correction through root cause analysis, pattern-based auto-repair with 234 error patterns, and continuous learning from outcomes.

[0045] HITL Gold Standard Library: A human-in-the-loop mechanism for capturing user feedback and promoting successful query-SQL pairs to become few-shot examples for LLM prompts.

[0046] Hybrid Template-LLM SQL Generation: A generation strategy where 78% of queries use validated templates and 22% use LLM generation, with hybrid refinement for medium-match cases achieving 99.1% success rate.

[0047] Trust Score Propagation: A multi-hop confidence propagation mechanism that reinforces successful mappings and demotes failed ones, achieving 97.8% accuracy in entity linking over time.

[0048] Automated Metadata Generation: An LLM-powered pipeline that transforms raw database DDL and sample data into production-ready semantic metadata in 2 hours versus 5-7 weeks of manual work.

[0049] **FIG. 1** illustrates an exemplary environment 100 of a system 103, which is configured to convert Natural Language Query to Structured Query Language (NQL-to-SQL) with automated content generation. In an embodiment, the environment 100 comprises, without limitation, a user input device 101, the system 103, a user interface 107, and a database 109. The system 103 and the user interface 107 may be connected via a predefined communication

network 105. The communication network 105, exemplified as any other suitable network facilitating communication between the system 103 and the user interface 107. The network 105 may include one or more wired and/or wireless networks.

[0050] In an embodiment, the user interface 102 is configured to receive a natural language query from the business user. The system comprises, without limiting to, an NLQ pre-processing module, a context discovery engine, an automated metadata generation module, a knowledge graph engine, and a SQL generation engine (which is not shown in FIG.1). In an embodiment, the natural language query is provided to an NLQ pre-processing module present in the system 103, which performs tokenization, intent extraction, and semantic entity identification. The processed query is transmitted to the context discovery engine, which is configured to perform constant-time lookup of relevant database columns, schema elements, aliases, and metadata using a hash-based or equivalent direct indexing mechanism. Thereafter, outputs of the context discovery engine are transmitted to the automated metadata generation module, which derives semantic metadata including data types, aggregation rules, temporal constraints, and relational context based on schema definitions, sample data, and inferred query intent.

[0051] In an embodiment, the generated metadata is processed by the knowledge graph engine, which stores a plurality of validated SQL query patterns and semantic relationships. The knowledge graph engine selects one or more applicable query templates corresponding to the identified query intent. Further, the LLM-based SQL generation engine receives a reduced and context-enriched input and generates a syntactically and semantically correct SQL query. The input comprises the selected metadata, validated query patterns, and minimal column information. Finally, the generated SQL query is stored in the database 109, and the resulting query output is returned to the user via the user interface 107.

[0052] FIG. 2 illustrates an exemplary flow diagram of a method for converting Natural Language Query to Structured Query Language (NQL-to-SQL) with automated content generation, in accordance with some embodiments of the present disclosure. In an embodiment, at block 201, the method comprises receiving, by a user, a Natural Language Query (NLQ) from a user via a user input device 101. At block 203, the method comprises processing the query through an automated metadata factory (Phase 1), which maintains pre-built indexes including a hash index, a vector database, a knowledge graph, and a constraint store. At block

205, the method comprises generating a Semantic Query Plan (SQP) as a database-agnostic intermediate representation. Upon generating the SQP, the method comprises synthesizing optimized SQL using a hybrid approach combining template matching and LLM-based generation with cost-aware model routing (at block 207). Further, at block 209, the method comprises executing the SQL and validating results through multi-layer validation.

[0053] FIG. 3 shows a high-level flow diagram illustrating the multi-phase pipeline architecture of the Natural Language Query to Structured Query Language (NQL-to-SQL) translation system, in accordance with some embodiments of the present disclosure. In an embodiment, the system 103 may be called an Adaptive Translation and Learning Analytics System (ATLAS), which implements a four-phase pipeline architecture for transforming natural language queries into accurate SQL queries with continuous learning capabilities. The phases are

Phase 0: Data Foundation,

Phase 1: Automated Metadata Factory,

Phase 2: Intelligent Query Understanding,

Phase 3: Context-Driven SQL Synthesis, and

Phase 4: Execution and Guardian Loop with feedback mechanisms enabling continuous improvement.

[0054] In an embodiment, the system 103 processes data through a plurality of sequential phases. The processing begins at the data foundation phase (Phase 0), and proceeds through the automated metadata generation phase (Phase 1), the intelligent query understanding phase (Phase 2), the SQL synthesis phase (Phase 3), and the execution and guardrails phase (Phase 4). The execution phase produces query results and derived insights. In addition, the system 103 includes a continuous learning component configured to receive feedback based on query outcomes and to iteratively update one or more of the preceding phases, as indicated by a feedback path connecting the continuous learning component to earlier phases of the system 103 as shown in **FIG. 3**.

[0055] FIGs. 4a and 4b illustrate an exemplary four-phase transformation summary diagram with feedback loops, in accordance with some embodiments of the present disclosure. In an embodiment, the data foundation phase includes one or more production databases, ETL and/or data pipelines, query and access logs, and user feedback signal sources. The automated

metadata generation phase includes modules for data profiling and statistics generation, business glossary and synonym management, semantic embeddings store, knowledge graph construction, constraint and business rule management, and hybrid metadata indexing. The query understanding phase includes modules for a query cache, an intent classification module, entity and metric extraction module, a synonym resolution module, a structured query plan (SQP) module, and a query validation module. The SQL synthesis phase includes modules for context retrieval, context pruning, constraint injection, a template-based SQL generator, an LLM-based SQL generator, and syntax and schema validation. Further, the execution and guardian loop phase includes modules for instrumented SQL execution, result parsing, result validation and anomaly detection, self-correction and automatic query repair, result intelligence and insights, Human feedback (Human-In-The-Loop (HITL) feedback), and learning and model updates as shown in **FIG. 4a**.

[0056] In an embodiment, phase 1 transforms raw schema into organized knowledge, providing instant lookup capability, elimination of manual documentation, smart synonym handling, and valid join path identification as shown in **FIG. 4b**. Phase 2 depicts the conversion of the natural language question into an understood intent, enabling ambiguity detection, caching mechanisms, multi-intent query handling, and database-agnostic processing. Phase 3 shows a validated SQP being converted into optimized SQL (constrained), achieving 82% cost reduction, hallucination elimination, template reuse, and pre-validated queries. Phase 4 illustrates SQL query transformation into validated results, ensuring no bad results, self-healing capabilities, and continuous learning. Further, feedback loops show that phase 4 updates phases 1, 2, and 3.

[0057] The key principles underlying each phase are tabulated in Table 1 below.

Phase	Idea
Phase 1	"Know everything about the database BEFORE anyone asks"
Phase 2	"Understand WHAT the user wants, not just what they said"
Phase 3	"Write SQL that's cheap, fast, and can't make mistakes"
Phase 4	"Validate everything, fix errors, and get smarter every time"

Table 1

Phase 1: Automated Metadata Factory

[0058] Phase 1 implements the Automated Metadata Factory that pre-processes the database schema into AI-optimized indexes. The main purpose of phase 1 is to perform expensive computation once and use it millions of times, achieving zero query-time overhead with instant retrieval capabilities.

[0059] **FIG. 5** illustrates an automated metadata generation pipeline for auto-creating metadata for new database tables, in accordance with some embodiments of the present disclosure. In an embodiment, the automated metadata generation pipeline comprises a five-stage workflow. The five-stages include a table sampler for fetching sample data, a schema analyzer for extracting metadata including column names, data types, and sample values. LLM metadata generator stage configured to generate YAML metadata with dimension keywords, metric formulas, business rules, and synonym mappings. In the LLM SQL example generator is configured to create few-shot examples with question-answer pairs. Further, the Auto-Save and Index Build for deploying metadata by appending to configuration files, rebuilding indexes, and reloading the system as shown in **FIG. 5**.

[0060] In an embodiment, the comprehensive system architecture for the Automated Metadata Factory depicts input sources including Production Databases, Schema Repository, ETL Logic sources, Query Samples, and User Feedback feeding into Data Profiling components including Schema Change Detection with Drift Monitor, Deep Profiling for Statistics and Patterns, LLM Context Generation for Automated Descriptions, Business Knowledge Extraction, and NLP Synonym Expansion (as shown in **FIG. 7**). The Semantic Intelligence layer contains modules for Semantic Embeddings, LLM Inference for Business Meaning, and Consistency Validation. The Relationships and Logic section features KPIs and Metrics Discovery, Knowledge Graph Builder with Temporal Knowledge Graph (TKG) and Trust Scores, and Formula Derivation for Constraints and Rules. The Hybrid Indexing and Retrieval layer contains Hash Index for O(1) Exact Match, Vector DB for Semantic Search, and Inverted Index for Fuzzy Matching, connecting to a Unified Knowledge Layer with Temporal Knowledge Graph and Metadata Store. Quality Assurance components include HITL Schema Verification and LLM Lineage Engine for Explainability.

[0061] In an embodiment, a deep data profiling engine module is configured to perform statistical profiling of database columns, semantic classification of data types, and domain detection. For each table, the system fetches sample data (typically 1,000 rows) and analyzes

column characteristics, including data types, value distributions, null percentages, and cardinality. In an embodiment, an LLM-powered automated metadata context generation module uses Large Language Models to automatically generate business descriptions, sample queries, and semantic context for each table and column. This eliminates the 5-7 weeks of manual documentation work traditionally required per table, reducing onboarding time to approximately 2 hours. In an embodiment, an intelligent synonym expansion module implements multi-source synonym discovery with context-aware disambiguation. Synonyms are extracted from historical query logs, external ontologies (ISO country codes, industry glossaries), and LLM inference. For example, "USA" is mapped to "UNITED STATES OF AMERICA" with associated confidence scores. In an embodiment, a semantic embedding generator module is configured to create hierarchical embeddings and query-metadata co-embeddings for semantic similarity search. The vector database stores 12,345+ embeddings, enabling similarity-based column discovery when exact matches are not found. In an embodiment, a semantic consistency and drift validation engine module is configured to detect and correct semantic drift, it flags inconsistencies caused by schema changes or learning updates and restores accurate semantic context.

[0062] In an embodiment, a Temporal Knowledge Graph (TKG) builder module is configured to construct a knowledge graph with schema versioning that captures entity relationships, valid join paths, and trust scores. The TKG contains 8,234 entities and 15,678 relationships, enabling intelligent join path discovery and preventing invalid table combinations. In an embodiment, trust scores calculation combines multiple signals such as schema guarantees (FK=1.0), value overlap (Jaccard similarity), historical success (query logs), and cardinality consistency. In an embodiment, a hybrid index builder module builds complementary index structures. In an embodiment, an automated metric discovery engine module automatically discovers KPIs and business metrics from ETL logic and query patterns, extracting formulas and aggregation rules. In an embodiment, a join semantics and negative knowledge engine module learns which tables may and/or may not be joined, storing negative rules such as "Never join TRANSACTIONS and MERCHANTS without common key" to prevent common errors. In an embodiment, a query pattern learner module is configured to analyze historical query logs to extract successful SQL patterns, building a library of 1,847+ validated templates with 99.2% success rates. In an embodiment, a metadata version control module maintains versioned metadata with drift detection and automatically triggers re-indexing when schema changes are detected.

[0063] In an embodiment, a hash index provides $O(1)$ constant-time exact keyword matching. The bidirectional hash map (keyword $\rightarrow$ columns, column $\rightarrow$ keywords) with synonym expansion layer enables 0.000-second lookups regardless of schema size (1K or 1M columns). In an embodiment, a vector database stores semantic embeddings for similarity search when exact matches are not found, containing 12,345 vectors for fuzzy matching capabilities. In an embodiment, the knowledge graph module maintains entity relationships and valid join paths with trust scores, preventing cartesian product errors and invalid joins. Further, the constraint store contains 456 business rules, data types, mandatory filters, and validation rules that are automatically injected into SQL generation.

[0064] In an embodiment, a governance, sensitivity and cost metadata engine module is configured to track data sensitivity levels, access controls, and query cost estimates for governance compliance. In an embodiment, a proactive metadata validation (HITL gateway) module, Human-in-the-loop validation enables domain experts to verify and correct auto-generated metadata before deployment. In an embodiment, a continuous learning and metadata evolution engine module continuously updates metadata based on query outcomes, user feedback, and schema changes, achieving 8.2% accuracy improvement over 3 months through adaptive index enrichment.

Phase 1 Inputs:

[0065] In an embodiment, the automated metadata factory receives one or more inputs (tabulated in Table 2). The one or more inputs include, without limiting to, a raw database schema, sample data, ETL/Transformation Code, Historical Query Logs and External Ontologies including ISO country codes, industry glossaries, and domain vocabularies.

Input	Description
Raw Database Schema	All tables, columns, data types (200+ tables, 3,500+ columns), VDC for table metadata, ETL upstream for complex logics for columns
Sample Data	1,000 rows per table for profiling
ETL/Transformation Code	Spark, SQL scripts from Git repos showing how data is derived

Input	Description
Historical Query Logs	Past user queries to learn synonyms and patterns
External Ontologies	ISO country codes, industry glossaries, domain vocabularies

Table 2

Phase 1 Outputs:

[0066] The phase produces four searchable indexes ready for Phases 2 and 3, achieving 99.8% schema coverage with real-time updates on schema changes. Specific outputs include, without limiting to the Hash Index, the Vector Database, the knowledge Graph, Synonym Dictionary, Constraint Store and LLM-Generated Context (shown in Table 3 below).

Output	Description
Hash Index	Exact keyword → column mapping (O(1) lookup)
Vector Database	Semantic embeddings for similarity search (12,345 vectors)
Knowledge Graph	Entity relationships, valid join paths, trust scores
Synonym Dictionary	"USA" → "UNITED STATES OF AMERICA" with confidence scores
Constraint Store	Business rules, data types, mandatory filters
LLM-Generated Context	Auto-generated descriptions, sample queries for each table

Table 3

Phase 1 Innovations:

[0067] Key innovations in phase 1 include an inverted index with multi-level synonym expansion, a Temporal Knowledge Graph (TKG) with schema versioning, and automated metadata generation from ETL upstream information and the principle of performing expensive computation once for millions of subsequent uses (Table 4).

Significance	Impact
Zero query-time overhead	All indexes pre-built, instant lookups
Eliminates manual documentation	LLM auto-generates business context from data + code
Enables synonym resolution	Users don't need to know exact column names
Prevents bad joins	Knowledge graph knows which tables can/cannot join
Self-updating	Refreshes automatically when schema changes

Table 4

[0068] FIG. 6 illustrates a comparative table evaluating three different approaches for Natural Language Query to Structured Query Language (NQL-to-SQL) conversion across capability metrics, in accordance with some embodiments of the present disclosure. In an embodiment, the comparative table evaluates three different approaches for natural language query to SQL conversion across capability metrics. The table compares the disclosed system against Semantic Model Solutions and Generic LLM (RAG) approaches across dimensions including Setup Time (2 hours automated vs. 5-7 weeks manual), Search Speed (O(1) hash achieving 0.000s vs. O(n) degradation), Token Efficiency (2-5 relevant columns vs. 50-100 embeddings), LLM Cost per Query (\$0.003 vs. \$0.01+), Pattern Reuse (150+ validated templates vs. none), Accuracy (95%+ vs. 85%), Scalability (infinite with 1M columns at same speed vs. degradation), and Business Rules (auto-applied vs. manual).

Phase 1 Significance

[0069] In an embodiment, without Phase 1, every query would need to scan the entire schema (slow), users would need exact column names (poor user experience), and joins would be guesswork (errors). The pre-built indexes eliminate query-time overhead, enable synonym resolution so users do not need to know exact column names, and prevent bad joins through knowledge graph validation.

[0070] In an embodiment, the Phase 1 intelligent metadata factory operates continuously prior to query execution and generates a versioned metadata repository. The metadata repository is configured to store structured and semantic information derived from one or more data sources

and to provide a query-ready knowledge base for use by downstream processing phases during query-time operations.

[0071] In an embodiment, Phase 1 processing operates as a foundational layer that executes continuously (24/7) and independently of user query arrival. Phase 1 is configured to transform raw database schema information into an intelligent and interconnected knowledge representation. The interconnected knowledge representation enables instantaneous semantic search through pre-computed vector embeddings, establishes trusted relationships among schema elements using a graph structure with associated confidence scores, and incorporates business rules awareness, including formulas and constraints extracted from underlying data sources. Phase 1 further supports synonym resolution by mapping expressions to canonical representations (for example, USA → UNITED STATES OF AMERICA) and enables pattern recognition based on analysis of historical query activity. By performing computationally intensive operations in advance, Phase 1 allows subsequent query processing to reuse the generated knowledge efficiently across a large number of queries.

[0072] FIG. 7 illustrates a flow diagram for an automated metadata factory (Phase 1) showing the complete data flow and processing pipeline, in accordance with some embodiments of the present disclosure. In an embodiment, the automated metadata generation module architecture comprises a plurality of functional components, including 13 core processing modules and four supporting systems. As shown in FIG. 7, the comprehensive sub-system architecture for the automated metadata factory depicts input sources including production databases, schema snapshots, ETL logic Git repository, query logs, and user feedback feeding into a data profiling layer. In an embodiment, the data profiling layer includes schema change detection with drift monitor, which receives input from schema snapshots, deep profiling for statistics and patterns, LLM context generation for automated descriptions, business description sample queries context, data lineage transform logic column relationship and synonym expander natural language mapping.

[0073] The Semantic Intelligence layer contains modules for Semantic Embeddings, LLM Inference for Business Meaning, and Consistency Validation. The Relationships and Logic section features KPIs and Metrics Discovery, Knowledge Graph Builder with Temporal Knowledge Graph (TKG) and Trust Scores, and Formula Derivation for Constraints and Rules. The Hybrid Indexing and Retrieval layer contains Hash Index for O(1) Exact Match, Vector

DB for Semantic Search, and Inverted Index for Fuzzy Matching, connecting to a Unified Knowledge Layer with Temporal Knowledge Graph and Metadata Store. Quality Assurance components include HITL Schema Verification and LLM Lineage Engine for Explainability.

Module 1: Deep Data Profiling Engine with LLM-Powered Automated Context Generation

[0074] In an embodiment, the module goes far beyond traditional "column has 195 distinct values" profiling. The module understands what those values mean. Process Steps include: Statistical Profiling (Count distinct values, nulls, min/max, distribution shape) and LLM-Powered Automated Context Generation. The LLM-Powered Automated Context Generation includes Multi-Source Input Collection comprising `SELECT * FROM table LIMIT 1000` (Sampled data rows), ETL Logic from Git Repo (Transform scripts, SQL views, data pipeline code), Schema metadata (DDL, constraints, indexes), and Existing documentation (Comments, wiki pages, README files). ETL Logic Analysis parses upstream transformation code to understand Data lineage (Where each column's data originates from), Business rules (Filters, joins, aggregations applied in ETL), Column relationships (How columns are derived/calculated), and Data quality rules (Validation logic, cleansing steps). Semantic Inference analyzes actual data values to identify data types, distributions, and business meaning. Business Description Generation creates human-readable descriptions for tables, columns, and relationships. Sample Query Generation automatically generates example queries that demonstrate common usage patterns. Context Enrichment outputs structured context (business descriptions, data types, sample values, query patterns, ETL logic) that becomes the foundation for all downstream modules.

[0075] In an embodiment, the process steps further include Semantic Classification (Is this a metric (numeric, aggregatable) or dimension (categorical, groupable)?), Domain Detection (Recognize geographic entities, dates, currencies, status codes), Quality Scoring (Calculate completeness, consistency, accuracy, timeliness), Usage Analysis (Which columns are frequently queried? Which operations are common (GROUP BY, WHERE)?), and Pattern Recognition (Detect hierarchies (Country → Region → Continent), time series patterns, seasonal trends).

[0076] In an embodiment, the module 1 provides a plurality of technical advantages arising from automated metadata generation and context-aware query processing. Module 1 is configured to perform zero-touch metadata generation, wherein LLM automatically creates

comprehensive schema documentation by analyzing actual data characteristics without requiring manual intervention. The generated business descriptions, example queries, and semantic annotations are subsequently utilized as contextual input during natural language query interpretation. The module further enables smart indexing by distinguishing between high-cardinality columns (`customer_id`) and low-cardinality (`country`) attributes. In guides query generation, the module knows to use `WHERE` on selective columns and `GROUP BY` on dimensions. Additionally, the system warns if the user tries to `SUM` a dimension or `GROUP BY` a metric. Performance optimization recommendations are dynamically generated based on actual usage query patterns, including recommendations for partition keys and indexes configurations. As a result, the system maintains a self-documenting metadata layer in which business-friendly descriptions and operational semantics are continuously updated.

LLM-Powered Automated Context Generation Engine

[0077] In an embodiment, the system 103 uses LLM to automatically analyze sampled data and generate a comprehensive business context without manual intervention. The processing steps include collecting data from multiple sources, including executing `SELECT * FROM table LIMIT 1000` to get a representative sample, parsing ETL scripts from Git repo (SQL, Python, Spark, dbt), extracting schema DDL and constraints, and gathering existing documentation. Thereafter, the process analyzes transformation code to extract source-to-target column mappings, business logic (CASE statements, formulas, aggregations), join relationships and foreign key logic and data quality rules (WHERE clauses, validation checks). Based on the collected data and analyzed transformation logic, a comprehensive contextual input is transmitted to the LLM model, the contextual input including sample data rows, schema metadata, ETL transformation logic, and existing documentation artifacts.

[0078] In an embodiment, the LLM returns business descriptions, semantic types, sample queries, and lineage explanations. The generated semantic metadata is validated by Cross-check LLM inferences against statistical patterns and ETL logic. Upon validation, the semantic metadata is stored in a metadata repository for subsequent retrieval and used for query-time retrieval.

Module 2: Intelligent Synonym Expansion

[0079] In an embodiment, the system 103 includes an intelligent synonym expansion module configured to map natural language expressions to canonical database values. The module

addresses variations between user terminology and underlying database representations, thereby enabling accurate query interpretation when user-provided terms differ from stored schema values. The intelligent synonym expansion module operates by executing a plurality of processing steps. The module analyzes historical mining by scanning query logs to identify commonly used user terms and expressions. Loading external ontologies and vocabularies, including standardized country codes, industry-specific terminology, and domain glossaries, is integrated to enrich the synonym space. Fuzzy matching logic is applied to resolve typographical errors and variations ("mercahnt" → "merchant"). Usage frequency analysis is performed to assign confidence weights to candidate synonyms based on observed user behavior (if 2847 users said "USA", high confidence). Contextual disambiguation logic evaluates surrounding query terms to distinguish between semantically similar concepts, for example, "volume" could mean payment_volume or transaction_volume - use surrounding words to decide. Multilingual mapping capabilities enable translation of terms across multiple languages, which supports 40+ languages with translation confidence. The module further incorporates a self-learning mechanism which updates confidence scores based on the success/failure after every query execution outcomes.

[0080] In an embodiment, the intelligent synonym expansion module provides multiple technical advantages, for instance, by allowing users to express queries using natural language rather than strict schema terminology. The module lowers the barrier to entry, that is, business users may not be required to memorize exact column names. Ambiguous terms are resolved through contextual analysis or interactive clarification. For example, the system may ask, "Did you mean payment volume or transaction volume?". Manual maintenance of synonym dictionaries is eliminated through automated learning from query usage. Over continued operation, the accuracy of term-to-schema mapping improves as the system refines confidence scores based on successful and unsuccessful query executions (Month 1 → 85% match rate, Month 6 → 97% match rate).

[0081] In an embodiment, consider an example, when a user submits a query containing a colloquial geographic reference, for example, "Show me fraud rate for USA", the module maps the reference to a corresponding canonical database value, without synonyms shows zero results (USA not in database). Thereafter, the module resolves associated metric terminology to an appropriate schema element, that is, Maps USA → UNITED STATES OF AMERICA, fraud rate → CY_FRAUD_RATE → SUCCESS.

Module 3: Semantic Embedding Generator

[0082] In an embodiment, a semantic embedding generation module is configured to transform metadata into vector representations suitable for semantic similarity search.

Module 3A: Semantic Inference & Meaning Extraction Engine

[0083] In an embodiment, a semantic inference and meaning extraction engine is provided and configured to derive business-level semantics. The engine utilizes LLM models to infer deep business semantics which are not explicit in schema metadata. Traditional metadata stops at column names and types. The module understands what data means in the business context, enabling the system to distinguish revenue columns from count columns automatically, detect measurement units (currency, percentage, count) without explicit labelling, understand encoded values (STATUS_CODE='A' means 'Approved') and link semantically related columns (AMOUNT always needs CURRENCY).

[0084] In an embodiment, the semantic inference and meaning extraction engine performs a sequence of steps includes (a) Column profiling data (cardinality, sample values, name patterns), (b) LLM semantic inference (GPT-4 analyzes context), (c) Business meaning extraction (metric vs dimension, units, semantics), (d) Cross-validation (verify against actual data patterns), (e) Confidence scoring (how certain are we?) and (f) Metadata enrichment (add semantic tags to knowledge base).

Column Semantic Inference Engine (LLM-Assisted)

[0085] In an embodiment, the system includes a column semantic inference engine which is assisted by the LLM model and configured to derive business semantics for database columns. The column semantic inference engine is configured to collect contextual information associated with a column name, data type, sample values and statistical characteristics derived from the underlying data. The collected contextual information is provided to the LLM model using a predefined prompt structure that instructs the large language model to infer a business meaning for the column based on the contextual information. The LLM prompt is “Given column CY_PAYMENT_VOLUME with type DECIMAL(31,0), average value 1.2M, max 450M, what is the business meaning?”. In response, the LLM returns a structured semantic output JSON with semantic type, unit, and a measurement type. Further, the module validates by cross-checking LLM inference against data patterns (for example, if LLM says "currency", verify values are positive numbers) and calculates a confidence score. Higher confidence score

if multiple signals agree (name contains "VOLUME", values are large integers, always positive).

[0086] In an embodiment, the system provides automatic semantic understanding without manual tagging by learning business meaning directly from data. The system matches user query intent to appropriate metrics or attributes, for example, when user asks "show me revenue", system knows to use CY_PAYMENT_VOLUME. Further, the system applies unit-aware formatting (currency gets \$, percentage gets %) to query results, and performs semantic validation to detect and prevent queries that are logically inconsistent with business rules (for example, "average of already-aggregated volumes").

Cross-Column Semantic Dependency Detector

[0087] In an embodiment, the module discovers hidden relationships between columns that always appear together or have logical dependencies. The module analyzes historical queries to find column pairs that appear together frequently and calculate correlation coefficients for numeric column pairs. Thereafter, performs value pattern analysis to detect conditional dependencies, that is, detect columns which always have values when another column has values (e.g., if AMOUNT > 0 then CURRENCY is always present). Subsequently, transmit discovered patterns to the LLM to validate if dependency makes business sense. Further, calculate confidence based on multiple signals and associated confidence scores are stored as semantic dependency relationships within a knowledge graph for use in subsequent query understanding and generation processes.

Module 3B: Semantic Consistency and Drift Validation Engine

[0088] In an embodiment, the module is to protect against silent semantic errors when data meaning changes without schema changes. The semantic drift detection module establishes a baseline semantic profile establishing normal meaning. The module continuously monitors incoming data patterns to evaluate statistical. Upon detection, the module analyzes to determine whether the deviation corresponds to a substantive change in semantic meaning. Thereafter, the LLM analysis validates if deviation is real semantic drift and creates a new semantic version after confirmation. Further, the module update queries to handle semantic change.

Semantic Consistency Validator

[0089] In an embodiment, a semantic consistency validator continuously validates the data values which matches with the expected semantic patterns. The process steps involves:

1. Load Semantic Profile: Expected value domains, patterns, constraints
2. Scan New Data: Sample recent data batches
3. Detect Anomalies: Values that violate semantic expectations
4. Classify Issues: Contradiction | Out-of-domain | Encoding inconsistency
5. Risk Assessment: Is this a data quality issue or semantic drift?
6. Alert + Remediation: Flag issues, suggest fixes

[0090] In an embodiment, the system improves analytical reliability by detecting and preventing silent data errors prior to query execution or result presentation. The system performs continuous data quality monitoring to ensure that generated metadata remains consistent with underlying data characteristics, and further supports automated remediation actions, including the application of corrective transformations to normalize or standardize data values.

Semantic Drift and Versioning Engine

[0091] In an embodiment, a semantic drift and versioning engine is configured to detect when column meaning changes over time without schema changes. The process steps are as follows:

1. Establish Baseline: Semantic fingerprint at time ‘T0’
2. Periodic Profiling: Re-profile column at regular intervals
3. Drift Detection: Compare the current semantic profile with baseline
4. LLM Validation: Confirm if drift is real semantic change vs. random variance
5. Impact Analysis: Which queries/analyses are affected?
6. Versioning Decision: Create new semantic version if confirmed
7. Temporal Validity: Mark when old meaning ended, new meaning began

[0092] In an embodiment, the system enforces semantic consistency over time by detecting changes in the definition or interpretation of metrics and preventing historical distortion arising from such changes. The system maintains temporal awareness of semantic definitions, automatically normalizes data across different semantic versions, and preserves an auditable history of semantic modifications to support compliance, traceability, and debugging.

Module 4: Temporal Knowledge Graph Builder

[0093] In an embodiment, the module is configured to build a relationship graph with temporal awareness and confidence scoring. A graph database where tables are nodes, relationships are

edges, and everything has a confidence score and temporal validity. The process steps are as follows:

1. Schema Extraction: Import declared foreign keys (100% confidence)
2. Relationship Discovery: Analyze value overlap to infer undeclared joins (87% confidence)
3. Semantic Linking: Connect columns with the same domain (ISSUER_COUNTRY ↔ MERCHANT_COUNTRY)
4. Temporal Modeling: Track state transitions (Customer snapshot Month 1 → Month 2)
5. Constraint Propagation: If table A requires filter X, propagate to joined tables
6. Graph Indexing: Build fast-lookup indexes (< 5ms single-hop, < 50ms shortest path)
7. Trust Calculation: Score every edge based on evidence strength

Module 5: Trust Score Calculator

[0094] In an embodiment, the trust score calculator module is configured to assign confidence scores to relationships based on evidence.

Module 6: Hybrid Index Builder

[0095] In an embodiment, the hybrid index builder module is configured to build three complementary indexes for O(1) exact, along with semantic and keyword search. The user's queries are represented in three different ways which require three indexes. A single index type may not handle all three efficiently. For example, the user queries in three different ways:

Exact: "Find column CY_PAYMENT_VOLUME" (knows exact name)

Semantic: "Show me revenue-related columns" (vague, natural language)

Keyword: "Find columns with 'fraud' in description" (partial text match)

[0096] In an embodiment, the process flow is as described below.

1. Hash Map Building: Create O(1) lookup for exact column names (8.4 MB, <1ms)
2. Vector Embedding: Generate 384-dimensional embeddings for all metadata (125 MB, built in 45 minutes)
3. HNSW Index: Build hierarchical graph for nearest-neighbor search (<10ms for top-10 results)
4. Inverted Index: Tokenize descriptions/comments, build keyword index (2.1 MB, <2ms)
5. Query Classification: Analyze incoming query to determine which index(es) to use
6. Result Fusion: If using multiple indexes, merge results using reciprocal rank fusion

[0097] In an embodiment, the output of the module is tabulated as shown in Table 5 below. The traditional systems choose one approach (SQL databases use B-trees, vector DBs use HNSW). The present disclosure uses adaptive routing - analyzes query to pick optimal index(es), gets best of all worlds.

Query Type	Index Used	Time	Result Quality
"CY_PAYMENT_VOLUME"	Hash only	0.8ms	100% (exact)
"Show revenue columns"	Vector only	8.3ms	94% recall
"columns with fraud"	Inverted only	1.7ms	89% recall
"payment volume"	Hybrid (all 3)	12ms	97% recall

Table 5

Module 7: Formula and Constraint Extractor

[0098] In an embodiment, the formula and constraint extractor module is configured to extract business logic from SQL views, stored procedures, and documentation.

Module 7A: Automated Metric Discovery Engine

[0099] In an embodiment, the module is configured to move individual columns to discover composite metrics, their calculation logic, and valid usage patterns. Users do not consider it in terms of columns, rather, user considered in business metrics form. For example, "Show me growth rate" means nothing to a column-centric system. The module discovers implicit metrics hidden in the data by computing metrics not stored as columns (growth rates, ratios, conversion rates), aggregating patterns (which columns should be SUM vs AVG vs COUNT), and validating dimensional breakdowns (can this metric be sliced by geography? time? product?). The process flow is described as follows:

1. SQL View Analysis → extract calculated fields
2. Query Log Mining → discover commonly computed metrics
3. Statistical Analysis → identify natural aggregation patterns
4. LLM Synthesis → understand business meaning of computations
5. Grain Detection → determine appropriate aggregation level
6. Dimension Validation → which dimensions make sense for this metric

Implicit Metric Discovery

[0100] In an embodiment, the module is configured to scan SQL views, query history, and BI reports to find metrics that aren't stored as columns. The process flow of the module is described as follows:

1. Parse SQL Views: Extract all computed expressions (CASE statements, arithmetic, aggregations)
2. Cluster Similar Computations: Group queries that calculate the same metric different ways
3. Normalize Formula: Standardize to canonical form
4. LLM Naming: Give metric a business name
5. Validate Pattern: Confirm metric makes business sense
6. Register Metric: Add to knowledge base with metadata.

Implicit Business Rule Extractor

[0101] In an embodiment, the module discovers hidden business rules that aren't enforced in schema but exist in data patterns. The process flow of the module is described as follows:

1. Pattern Detection: Scan data for conditional relationships
2. Rule Hypothesis Generation: LLM proposes potential rules
3. Statistical Validation: Verify rule holds across data
4. Confidence Scoring: How consistently does rule apply?
5. Rule Registration: Add to knowledge base
6. Query Integration: Use rules to validate/constrain queries

Module 7B: Join Semantics and Negative Knowledge Engine

[0102] In an embodiment, the module is to understand and join cardinality, safety and anti-patterns. Traditional metadata says "these columns have matching types" but do not understand cardinality: Is this 1:1, 1:N, or M:N? (M:N causes cartesian products), aggregation safety, that is, will this join change row counts in dangerous ways?, and forbidden Joins, for example, some joins are technically possible but analytically wrong. The module prevents join-related analytical corruption. The process flow of the module is described as follows:

1. Schema Analysis → extract declared foreign keys
2. Statistical Profiling → measure actual cardinalities
3. Query Log Mining → learn successful/failed join patterns
4. LLM Join Reasoning → understand business logic of joins
5. Cardinality Classification → 1:1, 1:N, N:1, M:N

6. Safety Scoring → aggregation-safe vs dangerous
7. Anti-Pattern Detection → discover forbidden joins from failures.

Join Semantics & Cardinality Inference

[0103] In an embodiment, the module is configured to automatically infer join cardinality and behavior without manual specification. The process flow of the module is described as follows:

1. Extract Join Candidates: Find columns with matching types/names
2. Sample Data Analysis: Measure actual cardinality by sampling joins
3. Statistical Classification: 1:1, 1:N, M:N based on row count behavior
4. Optional vs Mandatory: How many rows have matching values?
5. Aggregation Impact: Does join preserve/increase/decrease row counts?
6. Safety Assessment: Is this join safe for aggregations?

Negative Metadata (Anti-Patterns)

[0104] In an embodiment, the module learns from failures and explicitly stores what NOT to do. The process flow of the module is described as follows:

1. Failure Analysis: Parse errors from query history
2. Pattern Extraction: Identify common failure patterns
3. Forbidden Join Detection: Find joins that always fail/timeout
4. Deprecated Column Tracking: Mark obsolete columns
5. Unsafe Aggregation Registry: Record aggregations that produce wrong results
6. LLM Explanation: Understand WHY patterns are bad

Module 8: Query Pattern Learner

[0105] In an embodiment, the query pattern learner module is configured to learn common query patterns from history to optimize future queries.

Module 9: Metadata Version Control

[0106] In an embodiment, the metadata version control module is configured to track all metadata changes with full audit trail.

Module 10: Governance, Sensitivity & Cost Metadata Engine

[0107] In an embodiment, the purpose of the module is to get enterprise-grade compliance, security, and cost awareness for production deployment. Before any metadata goes to

production, the module is tagged for security, compliance, and cost impact. The process flow of the module is described as follows:

1. PII/PCI Detection: Scan column names, sample values, patterns to detect sensitive data
2. Sensitivity Classification: Label columns as PUBLIC, INTERNAL, CONFIDENTIAL, RESTRICTED
3. Masking Requirement Detection: Determine if columns need encryption, hashing, or redaction
4. Cost Profiling: Measure table sizes, scan costs, partition efficiency
5. Risk Classification: HIGH (>\$100/query), MEDIUM (\$10-\$100), LOW (<\$10)
6. Compliance Tagging: GDPR, CCPA, PCI-DSS, SOX applicable regulations
7. Access Control Mapping: Which roles can see this data?

Cost and Performance Metadata Annotator

[0108] In an embodiment, the cost and performance metadata annotator module is configured to measure and track query cost to enable cost-aware query generation.

Module 11: Proactive Metadata Validation (HITL Gateway)

[0109] In an embodiment, in this module, Human-in-the-loop (HITL) validation for low-confidence or high-impact metadata and provide the "Verified Badge" innovation. Not all AI-generated metadata is equally trustworthy. The module identifies what needs human validation.

The process flow of the module is described as follows:

1. Confidence Threshold Check: Is metadata confidence < 0.80?
2. Impact Assessment: Is this a critical object (core metric, frequently-used join)?
3. Validation Routing: Send low-confidence OR high-impact metadata to human reviewers
4. Expert Review: SMEs validate/correct AI-generated metadata
5. Verified Badge: Approved metadata gets badge, higher trust weight
6. AI Learning: System learns from human corrections
7. Prioritization: High-impact items reviewed first (core metrics before obscure columns)

Validation Dashboard (for human reviewers):



HIGH PRIORITY (15 items)
⚠ metric_payment_volume_growth Priority: 42.5 Confidence: 0.72 Impact: 847 queries/month Est. Time: 5 minutes [Review Now]
MEDIUM PRIORITY (34 items)
⚠ join_transactions_to_customers Priority: 28.3 Confidence: 0.78 Impact: 234 queries/month Est. Time: 3 minutes [Review]
LOW PRIORITY (127 items)
☑ Recently Verified: 89 items this week 🕒 Avg Review Time: 4.2 minutes 📊 Accuracy Improvement: +12% after human validation

Module 12: Confidence, Lineage and Explainability Layer

[0110] In an embodiment, the module verifies how metadata trustworthy and auditable by showing WHY metadata exists and HOW confident we are. Every piece of metadata must be explainable and traceable to its source. The process flow of the module is described as follows:

1. Source Tracking: Record where metadata came from (LLM, data analysis, human validation)
2. Evidence Collection: Store all signals that contributed to metadata (patterns, statistics, query logs)
3. Confidence Calculation: Combine evidence strength into single confidence score
4. Confidence Propagation: Flow confidence through graph edges (compound operations multiply confidences)
5. Lineage Graph: Track metadata dependencies (MetricA depends on ColumnB and ColumnC)
6. Explainability: Generate human-readable explanations of metadata decisions
7. Audit Trail: Complete history for regulatory compliance

Confidence Scoring and Propagation Engine

[0111] In an embodiment, the module is configured to assign a continuous score which flows through metadata relationships.

Metadata Lineage & Explanation Store

[0112] In an embodiment, every metadata object stores its complete provenance, that is, where it came from, how it was created, what evidence supports it.

Module 13: Continuous Learning and Metadata Evolution Engine

[0113] In an embodiment, the module is configured to learn continuously by every query, every validation, every schema change leads to system improvements. Traditional metadata systems are static. The module involves continuously learn from feedback. The process flow of the module is described as follows:

1. Feedback Collection: Gather signals from query success/failure, human validations, schema changes
2. Pattern Learning: Identify what feedback patterns mean (e.g., 5 synonym corrections for "USA" → update synonym confidence)
3. Metadata Updates: Automatically apply learned improvements
4. Trust Weight Adjustment: Boost/penalize metadata based on real-world performance
5. Deprecation Detection: Mark obsolete synonyms, patterns, rules
6. Model Retraining: Periodically retrain LLM prompts and embeddings
7. A/B Testing: Test metadata improvements on subset of queries before full rollout.

[0114] In an embodiment, upon completion of Phase 1, the system generates and stores a plurality of output artifacts shown in Table 6, each configured to support subsequent query understanding, context generation, and query synthesis operations. The success metrics are shown in Table 7 below.

Artifact	Size	Update Frequency	Used By
column_profiles.json	2.3 MB	Daily	Phase 2, 3
synonym_mappings.json	480 KB	Real-time	Phase 2
vector_embeddings/	125 MB	Weekly	Phase 2, 3
knowledge_graph.db	45 MB	Daily	Phase 3
trust_scores.json	1.2 MB	Real-time	Phase 3
inverted_index.json	2.1 MB	Daily	Phase 2
formulas.yaml	340 KB	Weekly	Phase 3
query_patterns.json	8.7 MB	Real-time	Phase 2

metadata_versions/	500 MB	Every change	All
schema_change_log.json	12 MB	Hourly scan	Phase 1, 4
drift_reports/	25 MB	Daily	Phase 1
sensitivity_tags.json	850 KB	Weekly	Phase 2, 3, 4
cost_metadata.json	1.1 MB	Daily	Phase 3
hitl_validation_queue.json	340 KB	Real-time	Human reviewers
confidence_scores.json	2.8 MB	Real-time	All phases
metadata_lineage/	180 MB	Per change	Audit/Debug
evolution_history.json	45 MB	Daily	Analytics

Table 6

Metric	Target	Current	Status
Metadata Coverage	100% of production tables	98.7%	✓
PII Detection Accuracy	> 95%	97.2%	✓
Cost Prediction Accuracy	> 90%	94.1%	✓
HITL Validation Rate	5-10% of metadata	7.3%	✓
Verified Metadata Coverage	> 80% high-impact	82.6%	✓
Confidence Propagation Accuracy	> 85%	88.4%	✓
Learning Loop Improvement Rate	+10% QoQ	+12.3%	✓
Metadata Lineage Completeness	100% trackable	99.1%	✓
Embedding Freshness	< 7 days	3 days	✓
Trust Score Avg	> 0.90	0.96	✓

Index Build Time	< 60 min	45 min	✓
Pattern Library Size	> 500 patterns	847 patterns	✓
Synonym Accuracy	> 95%	97.2%	✓
Schema Change Detection Latency	< 1 hour	12 min	✓
Drift Detection Accuracy	> 90%	94.3%	✓

Table 7

[0115] In an embodiment, upon completion of the automated metadata generation phase (Phase 1), a query understanding phase (Phase 2) is initiated. The query understanding phase is configured to perform semantic search over vector embeddings, resolve terms using synonym mappings, traverse a knowledge graph to identify relational paths, match the query against previously learned patterns for optimization, and apply reference trust scores to select appropriate join selection. The automated metadata generation phase (Phase 1) operates continuously in the background to update the semantic context, while the query understanding phase and subsequent processing phases (Phases 2-4) are executed on-demand in response to individual user queries.

[0116] In an embodiment, the intelligent metadata factory enables creation of a self-evolving, semantically-aware system which prevents analytical errors before they occur.

Concepts of Phase 1

[0117] A method for semantic metadata enrichment comprising: (1) profiling database columns to extract statistical patterns, (2) feeding patterns to LLM for business meaning inference, (3) validating LLM inference against actual data distributions, (4) calculating confidence scores from multi-signal agreement, (5) storing validated semantics in queryable knowledge base.

[0118] A system for discovering semantic column dependencies comprising: (1) mining query logs for column co-occurrence patterns, (2) calculating statistical correlations between column pairs, (3) detecting null-dependency patterns (when col1 has value, col2 must have value), (4)

LLM validation of discovered patterns for business logic consistency, (5) creating enforceable "use-together" rules in query validator.

[0119] A method for semantic drift detection and versioning comprising: (1) establishing baseline semantic profile (value domain, distribution, units), (2) continuous monitoring of semantic characteristics, (3) LLM-validated detection of meaning changes, (4) creation of semantic versions with temporal validity, (5) automatic query adaptation for time-series spanning semantic transitions, (6) impact analysis of semantic changes on historical queries.

[0120] A system for automated metric discovery comprising: (1) parsing SQL views and stored procedures to extract computed expressions, (2) mining query logs for frequently calculated formulas, (3) clustering similar computational patterns using normalized formula representation, (4) LLM-based naming and semantic classification of discovered metrics, (5) statistical analysis to determine aggregation behavior (additive vs non-additive), (6) validation of dimensional compatibility (which GROUP BY clauses make sense), (7) registration in metric catalog with usage examples.

[0121] A method for extracting implicit business rules comprising: (1) statistical analysis of conditional value patterns in data, (2) detection of temporal and referential constraints not enforced in schema, (3) LLM validation of discovered patterns for business logic plausibility, (4) classification of enforcement levels based on violation rates and business impact, (5) registration of rules in query validator for proactive error prevention.

[0122] A system for automated join characterization comprising: (1) statistical sampling of join operations to measure row count changes, (2) classification of cardinality relationships based on row multiplication patterns, (3) calculation of aggregation safety scores considering row explosion factors, (4) detection of optional vs mandatory join relationships from null rate analysis, (5) generation of safe aggregation guidance (aggregate-before-join recommendations for M:N relationships).

[0123] A system for learning and preventing anti-patterns comprising: (1) automated analysis of failed query logs to extract common failure patterns, (2) classification of failure types (cartesian products, deprecated columns, semantic errors), (3) LLM generation of explanatory error messages and alternative approaches, (4) proactive query validation against anti-pattern repository, (5) continuous learning from new failures to expand prevention rules.

[0124] A multi-modal metadata indexing system comprising: (1) hash map for exact identifier matching, (2) HNSW vector index for semantic similarity search using embeddings, (3) inverted index for keyword search, (4) query classification engine that routes to optimal index(es), (5) reciprocal rank fusion algorithm that merges results from multiple indexes with confidence weighting.

[0125] A self-healing metadata system comprising: (1) continuous schema monitoring with change detection, (2) similarity-based inference of column renames vs removals, (3) automated remediation workflows (profile → embed → index → validate), (4) impact analysis to identify affected queries and patterns, (5) graceful deprecation with warnings instead of immediate failures, (6) change log integration with query execution for temporal query adaptation.

[0126] A system for semantic drift detection comprising: (1) continuous profiling of column distributions (cardinality, null rates, KL divergence), (2) detection of statistically significant distribution changes, (3) LLM-based validation to distinguish semantic drift from random variance, (4) automated remediation (re-profiling, synonym updates, pattern invalidation), (5) severity classification and alert generation for manual review when needed.

[0127] A dynamic trust scoring system comprising: (1) multi-factor trust calculation from schema enforcement, statistical value overlap, historical query success, and cardinality validation, (2) temporal decay function reducing trust for stale relationships, (3) error-based trust adjustment with severity weighting, (4) confidence propagation through knowledge graph for transitive relationships, (5) trust threshold-based join recommendation (high-trust joins prioritized in query generation).

[0128] A method for formula extraction and normalization comprising: (1) SQL parsing to extract computed expressions from views and stored procedures, (2) normalization to canonical formula representation using algebraic equivalence rules, (3) separation of null handling logic from core calculation, (4) LLM-based semantic validation and naming, (5) registration in formula library with required columns, null handling, and usage constraints.

[0129] A query pattern learning system comprising: (1) clustering of historical queries into parameterizable templates, (2) extraction of common intent patterns (corridor analysis, time series, ranking), (3) statistical modeling of performance characteristics per template, (4)

predictive modeling of execution time based on query features, (5) anti-pattern detection from failure rates, (6) automatic optimization recommendation based on successful pattern variants.

[0130] A metadata version control system comprising: (1) git-like versioning with commit-style changelogs, (2) automated impact analysis predicting affected queries per change, (3) rollback capability with estimated recovery time, (4) diff visualization showing metadata changes between versions, (5) A/B testing framework for metadata experiments, (6) change approval workflows with impact review.

[0131] A self-improving synonym system comprising: (1) multi-source synonym discovery from query logs, domain ontologies, fuzzy string matching, and LLM inference, (2) confidence scoring from multiple evidence signals, (3) reinforcement learning from query success/failure to adjust confidence, (4) context-aware disambiguation using query semantic analysis, (5) multilingual synonym mapping with language-specific confidence, (6) automatic deprecation of low-confidence synonyms

Phase 2: Intelligent Query Understanding and Semantic Planning

[0132] In an embodiment, the intelligent query understanding and semantic planning module is configured to transform natural language into validated, database-agnostic Semantic Query Plans (SQP) with intelligent caching and early error detection. The average latency for phase 2 is 75ms or less than 100ms.

[0133] In an embodiment, Phase 2 comprises a real-time processing phase that is initiated upon receipt of a natural language query submitted by a user. In this phase, the system processes the query through a structured processing pipeline configured to interpret user intent and convert the natural language input into a structured database query. The pipeline bridges the semantic gap between business-level language used by the user and the formal query structure required for database execution.

What Phase 2 Does:

[0134] In an embodiment, phase 2 decodes the intent of the user query using a Natural Language Processing (NLP) and Large Language Model (LLM) hybrid approach to determine the actual analytical objective beyond simple keyword matching. Also, phase 2 performs intelligent caching by querying a multi-level semantic cache to retrieve previously computed results, thereby enabling rapid response generation with high cache hit probability and reduced

latency (40%+ hit rate, sub-10ms response). Phase 2 performs entity extraction to identify relevant metrics, dimensions, filtering conditions, and temporal constraints (94% F1 score) from the user query. In addition, synonym resolution is carried out to map business language/terminology, abbreviations, typographical variations, and multilingual expressions to canonical database values. Based on the interpreted intent and extracted entities, the system generates the SQP represented as a database-agnostic logical query plan in an intermediate structured format (JSON intermediate representation). Prior to query translation, the system performs early validation to detect inconsistencies or logical errors in the interpreted request, thereby preventing unnecessary computational overhead associated with failed query execution (for example, saves 85% of failed query costs). Further, when the confidence score of the interpreted intent falls below a predefined threshold (that is, below < 0.80), the system performs ambiguity detection and automatically generates clarifying prompts to obtain additional input from the user before proceeding with query generation.

[0135] In an embodiment, phase 2 of processing is implemented as an intermediate semantic interpretation stage between natural language input and SQL generation. This phase enables caching of structured query intent representations, thereby reducing repeated high LLM token usage and computational cost. Phase 2 allows early detection and correction of interpretation errors prior to SQL generation, improving reliability. Additionally, the intermediate representation remains independent of any specific database schema, enabling support for multiple databases. Further, phase 2 enhances transparency and debuggability by allowing validation of the semantic representation before final query generation (black box from text→SQL).

[0136] In an embodiment, Phase 2 implements the SQP comprising a structured intermediate representation configured to facilitate semantic-level query processing. The SQP enables intelligent caching based on semantic equivalence rather than exact text matching, thereby improving query reuse efficiency. The SQP allows early validation of query elements against Phase 1 to ensure structural and schema consistency prior to SQL generation. Additionally, the SQP supports the generation of SQL statements compatible with multiple SQL dialects, including BigQuery, PostgreSQL, and Snowflake, from the same intermediate representation. The SQP also provides a clear debugging mechanism by enabling inspection of an SQP JSON representation before SQL translation, thereby improving transparency of query construction. Further, the reuse of SQP templates reduces LLM token consumption by approximately sixty

percent. Thus, the SQP acts as a "semantic compiler IR" - similar to how programming language compilers use intermediate representations (LLVM IR, Java bytecode) to separate frontend parsing from backend code generation.

[0137] FIG. 9 illustrates a flow diagram for Query Understanding (Phase 2) showing the complete query processing pipeline, in accordance with some embodiments of the present disclosure. The phase 2 architecture comprises ten core modules configured to perform primary functional operations of the system, in combination with a multi-level cache architecture. As illustrated in **FIG. 9**, the detailed architecture shows a User Input Layer with user context elements and natural language query input flowing into Query Pre-Processing and Validation containing modules for Intent Classification with Multi-Intent Detection, Query Complexity Analysis and Risk Assessment, and Ambiguity Detection with Confidence Scoring. The Intelligent Query Understanding section contains L1 Exact Fingerprint matching, L2 Parametric Template matching, and L3 Semantic Similarity search connecting to cached result storage. The Entity Extraction and Resolution section contains Named Entity Recognition, Contextual Entity Linking and Column Binding, Synonym Resolution with Fuzzy Matching, and Temporal Expression Parsing. The Semantic Query Planning section contains SQP Generation, Template Matching, and Join Path Planning with TKG Traversal. The Multi-Layer Validation section includes Schema Validation, Semantic Validation, Performance Validation, and Security Validation, with outputs to Valid SQP for Phase 3 or Clarification Request for user interaction.

Module 1: Intelligent Intent Classification Engine:

[0138] In an embodiment, an intelligent intent classification engine module (M1) detects the query type, including metric retrieval, growth calculation, ranking, comparison, and aggregation. The module is configured to decode user's true intent from natural language with multi-intent detection and confidence scoring. The system achieves 93.7% intent classification accuracy through multi-signal analysis of query structure, keywords, and context. The module uses a hybrid NLP and LLM approach to classify intent with high confidence while detecting ambiguity. Further, a series of process steps executed by the module are:

1. Linguistic Analysis: Parse sentence structure (subject, verb, object, modifiers)
2. Keyword Extraction: Identify intent signals ("show", "compare", "trend", "top")
3. Pattern Matching: Match against 200+ known query patterns
4. LLM Classification: Use GPT-4 for ambiguous cases (15% of queries)

5. Multi-Intent Detection: Handle compound queries ("show volume AND fraud rate")
6. Confidence Scoring: Calculate certainty (0.0-1.0) based on signal strength
7. Context Integration: Use user history + current dashboard to disambiguate

[0139] In an embodiment, identification and clarification of user intent are critical for ensuring accurate and efficient system operation. By distinguishing between similar but contextually different requests, such as requesting the “top 10” results versus the “bottom 10” results, the system prevents potential misinterpretation of the user’s query. The clarity further enables early routing of the request within the processing architecture, as different types of intents may require distinct execution paths or computational workflows. Additionally, accurate intent identification guides the level of validation required during query processing. For instance, simple metric retrieval operations typically require less validation compared to more complex operations involving multiple data joins or aggregations. Consequently, by assessing the confidence level associated with the interpreted intent, the system may enhance the overall user experience. For example, when the confidence level is high, the system proceeds with execution, whereas when the confidence level is low, the system prompts the user for clarification to ensure correct interpretation before processing the request.

[0140] In an embodiment, consider an exemplary example in which a user submits a query requesting to “show payment volume.” In the absence of contextual information, the system may interpret the request in multiple ways, such as retrieving the total payment volume through an aggregation operation or generating a breakdown of payment volume by country through a grouping operation, thereby resulting in an interpretation confidence of approximately 50%. However, when contextual information associated with prior user interactions is considered, such as earlier queries requesting country-wise payment breakdowns, the system determines that the present query is more likely intended to produce grouped results. Consequently, the system infers that the query corresponds to a GROUP BY operation with an increased interpretation confidence, for example, about 92%, thereby enabling the system to execute the query directly without issuing an additional clarification request to the user.

Module 2: Query Complexity Analyzer and Risk Assessor

[0141] In an embodiment, a multi-intent detector (or a query complexity analyzer and risk assessor) module (M2) is configured to predict query complexity, execution cost, and failure risk before proceeding with expensive processing. The module handles compound questions

such as "Show payment volume AND fraud rate" by detecting multiple intents and enabling parallel execution with smart result merging. For example, "Show payment volume for USA in Sep 2024" is a simple query (uses 1 table, 3 filters, 1 metric) and "Show YoY growth by country for top 10 corridors with fraud rate correlation" is a complex query (uses multiple tables, growth calculation, ranking, correlation). The complexity analyzer predicts query difficulty using machine learning-based models trained on approximately 50,000 historical queries to analyze and evaluate incoming queries prior to execution. In an embodiment, the module is configured to compute a complexity score within a predefined range of 0.0 to 1.0 based on the structural and semantic components of the query. The module predicts an estimated execution cost comprising at least a database scan cost and the LLM token processing cost. The module additionally estimates an expected latency corresponding to a projected execution time required for processing the query. In addition, the module performs a risk assessment to determine a probability of execution failure or timeout associated with the query. Further, the module performs a routing operation, wherein relatively simple queries are directed to a fast execution path, while relatively complex queries are routed to an enhanced validation or processing pathway to improve reliability and system performance. By assessing the complexity of incoming queries, the system may perform effective resource planning, ensuring that high-complexity queries are prioritized in queued for processing. Also supports user expectation setting by providing advanced notifications, such as informing the user which the processing may take approximately 5–8 seconds. Additionally, the system facilitates early rejection of queries which are likely to fail or incur high computational cost, preventing unnecessary resource consumption. Further, the system enables adaptive processing, wherein simple queries can bypass computationally expensive LLM operations and instead be handled using predefined templates, thereby improving overall efficiency and reducing latency.

Module 3: Ambiguity Detector and Confidence Scorer

[0142] In an embodiment, an ambiguity detector and confidence scorer module (M3) is configured to detect ambiguous queries and decide whether to proceed with high confidence or ask for clarification. The module implements confidence scoring on a 0.0-1.0 scale, triggering clarification requests when confidence falls below 0.80 threshold. This catches ambiguity early, asking clarifying questions before writing incorrect SQL. The system achieves 83.2% ambiguity detection precision.

[0143] In one embodiment, the module recognizes certain queries which may contain multiple possible meanings, for example, a request such as “show volume” may correspond to payment volume, transaction volume, or fraud volume, and may further represent either an aggregated total volume or a grouped breakdown volume across different time periods. To address such ambiguity, the module computes a confidence score for each possible interpretation using a multi-signal analysis framework. The analysis includes evaluation of intent pattern matching to determine the degree to which the query corresponds to known intent structures (0.0 – 1.0 score), entity recognition confidence to assess the certainty of extracted entities, synonym resolution quality to measure the strength of semantic mappings, context alignment with prior user history and/or dashboards, and historical success rates of similar queries (succeeded or failed). Based on the calculated confidence score, the system applies a decision logic in which interpretations with a confidence score equal to or greater than 0.85 are executed automatically, interpretations with a confidence score between 0.65 and 0.85 are executed with a low-confidence notification, and interpretations with a confidence score below 0.65 trigger a clarification request to the user to resolve the detected ambiguity.

[0144] In an embodiment, the system prevents potential errors by avoiding assumptions when the intent is uncertain and significantly reduces failed queries (67% of failed queries), thereby improving overall accuracy and operational stability. At the same time, the system improves user experience by requesting clarification only when it is genuinely necessary (76.8% acceptance rate). Further, the system is designed to continuously improve over time by updating the confidence thresholds based on observed success patterns.

[0145] In an embodiment, an entity extraction pipeline comprises four integrated modules.

- (a) Named Entity Recognition Module (M4): Extracts metrics, dimensions, filters from the natural language query, modifiers, temporal, and operators.
- (b) Contextual Entity Linker and Column Binder Module (M5): Links extracted entities to specific database columns using the Phase 1 knowledge graph, achieving 87% probability scoring accuracy.
- (c) Synonym Resolution and Fuzzy Matching Module (M6): Resolves synonyms using the hash index with confidence propagation through multi-hop chain expansion.
- (d) Temporal Expression Parser and Normalizer Module (M7): Parses temporal expressions like "last month" or "Q3 2024" and normalizes them to specific date ranges with automatic granularity detection (daily/monthly/quarterly).

Module 4: Named Entity Recognition and Extraction

[0146] In an embodiment, a named entity recognition and extraction module is configured to identify and extract metrics, dimensions, filters, and temporal constraints from natural language. Once intent is classified and confidence is acceptable, the module extracts the concrete entities which define what data to query. The entity types include (a) metrics, representing quantitative measures such as payment volume, fraud rate, and transaction count, (b) dimensions, representing categorical attributes used for grouping results such as country, merchant category, or product type, (c) filters, representing constraints applied to narrow the result set such as geographic regions (e.g., USA), specific periods (e.g., September 2024), or transaction characteristics (e.g., cross-border only), (d) modifiers, representing transformations applied to the retrieved data including growth computation, ranking, or aggregation, (e) temporal entities, representing time constraints including explicit dates, date ranges, or relative periods, and (f) operators, representing relationship indicators which define associations between entities, including terms such as “to,” “versus,” “by,” or “per”.

[0147] In an embodiment, the system processes the query through a structured multi-stage pipeline. Initially, a tokenization stage decomposes the input query into meaningful phrases or tokens to facilitate downstream analysis. Subsequently, a part-of-speech (POS) tagging stage identifies grammatical roles within the tokens, including nouns corresponding to entities, verbs corresponding to actions, and adjectives corresponding to modifiers. Thereafter, a dependency parsing stage determines syntactic relationships between the tokens to establish contextual associations among entities. The system further performs named entity recognition to classify identified terms into predefined entity categories such as metric, geographic entity, or temporal entity. Upon entity identification, a vector search mechanism maps the extracted entity text to canonical database column names using semantic similarity. The vector search achieves approximately a 94% F1 recognition score, enabling reliable entity normalization. Further, the system performs confidence scoring for each extracted entity, assigning a certainty value within a range of 0.0 to 1.0. Thus, the module improves analytical query interpretation by providing high precision in entity recognition compared with traditional keyword matching techniques. For example, the system is capable of recognizing variations such as “USA,” “United States,” “US,” and “America” as representing the same geographic entity. Additionally, the system supports the extraction of multiple metrics and dimensions from complex queries while

maintaining confidence tracking for each identified element, thereby enabling accurate query translation and improved downstream data analytics processing.

Module 5: Contextual Entity Linker and Column Binder

[0148] In an embodiment, a contextual entity linker and column binder is configured to bind extracted entities to specific database columns using a knowledge graph and context analysis. After entities are extracted, the queries may be linked to the correct database columns. This is harder than it seems because same entity could map to multiple columns (e.g., "USA" → ISSUER_COUNTRY or MERCHANT_COUNTRY?), multiple synonyms for same column (e.g., "fraud", "fraudulent", "fraud rate" → CY_FRAUD_RATE), and context determines correct mapping (e.g., "volume" → payment_volume in payment context, card_count in card context).

[0149] In an embodiment, the module process is performed to accurately map an entity to a corresponding data column. The process includes generating candidate columns by identifying all columns in which the entity value exists. Thereafter, a vector search is performed to determine semantic similarity between the entity text and column descriptions. Subsequently, context scoring is applied using the query context, including user intent, other identified entities, and operators, to score the candidate columns. The process also includes performing a knowledge graph lookup in a Table Knowledge Graph (TKG) to evaluate column relationships and usage patterns, followed by checking historical usage data to determine whether the entity has previously been successfully associated with particular columns. Based on the aggregated scores obtained from the above steps, the column having the highest score is selected as the best match when the confidence score exceeds a predefined threshold of 0.75. The process enables effective disambiguation of entities with an accuracy of approximately 94%, allowing context-aware mapping in which the same entity may correspond to different columns depending on the query context. Further, utilizes relationship metadata from the TKG for validation, and continuously improves over time through learning from user corrections.

Module 6: Synonym Resolution and Fuzzy Matching

[0150] In an embodiment, a synonym resolution and fuzzy matching module is configured to map business language, typos, and abbreviations to canonical database values. In practical scenarios, users typically express queries using common or conversational terms rather than formal database column names. For example, a user may refer to "US" instead of "UNITED STATES OF AMERICA", "fraud" instead of a system metric such as "CY_FRAUD_RATE",

or “last month” instead of a specific time identifier such as “202411”. Thus, the module operates to bridge the semantic gap between natural language user input and structured database terminology.

[0151] In an embodiment, the module supports multiple synonym categories. The synonyms include exact synonyms, in which a direct one-to-one mapping (1:1 mapping) is maintained between commonly used terms and canonical database representations, such as mapping “USA” to “UNITED STATES OF AMERICA”. The module also handles abbreviations through multi-hop synonym chains, enabling progressive resolution of abbreviated forms, for example, “US” mapped to “USA” and subsequently to “UNITED STATES OF AMERICA” (multi-hop). Additionally, the module accommodates typographical errors and misspellings by applying fuzzy-matching techniques, such as mapping “paymnet volme” to “payment volume”. Multilingual mappings are also supported, allowing equivalent terms in different languages, for instance, mapping “Estados Unidos” to “UNITED STATES OF AMERICA”. Further, domain-specific synonyms are resolved using contextual interpretation, enabling terms such as “fraud” to correspond to domain-specific metrics such as “CY_FRAUD_RATE”. The resolution process follows a hierarchical strategy to ensure both efficiency and accuracy. Initially, the module performs an exact match lookup against a synonym dictionary, enabling constant-time retrieval of known mappings. In the absence of an exact match, the system performs fuzzy matching based on edit distance and similarity metrics, for example, applying a Levenshtein distance threshold of less than two and a similarity score greater than 0.80. Where abbreviation chains are detected, the system performs multi-hop resolution by sequentially traversing linked synonym mappings until a canonical entity is identified. In situations where a term may correspond to multiple meanings, contextual resolution is performed using query context, allowing a term such as “volume” to be interpreted as “payment_volume” in a payment context or “card_count” in a card management context. The module further incorporates a historical learning mechanism in which user feedback and corrections are recorded and used to update confidence scores associated with synonym mappings.

Module 7: Temporal Expression Parser and Normalizer

[0152] In an embodiment, a temporal expression parser and normalizer module is configured to parse and normalize temporal expressions into database-compatible formats. Users express time in many ways, for example, “last quarter”, “Sep 2024”, “YTD”, “last 12 months”. The

module converts all temporal expressions into normalized database formats. The supported expressions are absolute Dates, for example, "Sep 2024" → 202409, "2024-09-15" → 20240915, relative Periods: "last month" → 202411, "last quarter" → [202407, 202408, 202409], fiscal Periods: "Q3 2024" → [202407, 202408, 202409], rolling Windows: "last 12 months" → [202312, 202401, ..., 202411], year-to-date: "YTD" → [202401, 202402, ..., 202412] and special keywords: "current", "previous", "latest". In an embodiment, the process includes recognizing the temporal pattern by identifying whether the expression represents an absolute, relative, or range-based time reference. Thereafter performs context resolution, wherein relative expressions such as "last month" or "current month" are interpreted based on the present system date. Subsequently, granularity detection is carried out to determine the appropriate temporal level, including monthly, daily, quarterly, or yearly intervals. Further, the identified temporal information is normalized into a standardized database format, such as YYYYMM or YYYYMMDD, to enable consistent data retrieval. Finally, a validation step is performed to verify whether the identified temporal period exists within the database. As a result, the process enables natural language interaction by allowing users to specify time periods in common expressions such as "last quarter" rather than explicit numerical formats, say "[202407, 202408, 202409]". Further, the process ensures context-aware interpretation based on the current date, automatically detects the appropriate temporal granularity, and performs validation to notify users when the requested time period does not contain available data.

Module 8: Semantic Query Plan (SQP) Generator

[0153] In an embodiment, a Semantic Query Plan (SQP) generator module is configured to create/generate a database-agnostic semantic plan/logical query plan in JSON format that serves as an intermediate representation (IR) similar to compiler design and validate entities. The SQP captures intent, metrics, filters, time ranges, aggregations, and confidence scores without being tied to any specific database dialect. After all entities are extracted, validated, and linked, the module assembles them into an SQP, a structured JSON representation which describes WHAT to query (not HOW in SQL). The SQP components include, without limiting to, intent: What type of query (metric_retrieval, comparison, growth, ranking), Metrics: Which quantitative measures with aggregations, Dimensions: Grouping attributes, Filters: WHERE clause constraints, Grain: Level of detail (country-month vs country-day), Join Plan: How to connect multiple tables (if needed), Order/Limit: Sorting and result size constraints, and Metadata: Confidence scores, provenance, optimization hints. This approach is database-

agnostic, allowing the same SQP to generate SQL for BigQuery, PostgreSQL, or Snowflake. It enables early error detection before SQL generation, supports template reuse for 34.7% of queries, allows optimization of the logical plan, and facilitates debugging by inspecting the query interpretation in JSON.

Module 8a: SQP Template Matcher and Parameter Binder

[0154] In an embodiment, an SQP template matcher and parameter binder module is configured to match queries to pre-validated SQP templates and bind parameters instead of generating them from scratch. In many applications, queries often follow recurring patterns, and leveraging this regularity can significantly optimize performance and cost. To achieve this, the module first extracts a query signature by identifying the intent, metric types, and dimension types associated with the query. It then searches a library of over 200 pre-validated SQP templates to find suitable candidates. A fuzzy matching step ensures that minor variations in the query are accommodated, typically requiring at least 80% similarity to a template. Once a matching template is identified, parameters from the query are bound to the template placeholders, followed by a quick validation step to ensure that all bound parameters are correct and consistent. The resulting SQP is then returned, achieving a processing speed approximately 45 milliseconds faster than generating the query via an LLM. This approach offers multiple advantages, that is, roughly three times faster than full LLM generation, ensures high accuracy with a 99.2% success rate due to the use of pre-validated templates, reduces operational costs by eliminating LLM token usage (saving approximately \$0.015 per query), and maintains consistency, as the same query pattern always produces the same SQP structure.

Module 8b: Join Path Planner with Temporal Knowledge Graph (TKG) Traversal

[0155] In an embodiment, a join path planner with a TKG traversal module is configured to determine an optimal join path when a query requires multiple tables using the Temporal Knowledge Graph. When a query requires data from multiple tables, such as “Show payment volume by customer segment,” the process begins by identifying the necessary required tables, in this case, TRANSACTIONS, which contains payment_volume, and CUSTOMERS, which contains segment. Next, the module queries the knowledge graph (TKG) to discover all possible paths connecting TRANSACTIONS to CUSTOMERS. Each path is then scored based on metrics such as trust score, cardinality, and cost. The optimal path is selected by prioritizing the highest trust relationships and the lowest cost in terms of performance and resource usage. After path selection, any required constraints are validated, ensuring that mandatory filters are

applied and that the relationships adhere to proper cardinality rules. Finally, a join plan is generated, specifying the join columns, join type, and the order of joins. This approach is critical for preventing low-trust or incorrect joins, ensuring query correctness, optimizing performance by leveraging indexed columns and reducing intermediate results, and maintaining accurate cardinality handling between tables, such as distinguishing between one-to-many and many-to-many relationships.

Module 8c: SQP Optimizer with Logical Rewrite Rules

[0156] In an embodiment, an SQP optimizer with a logical rewrite rules module is configured to optimize SQP before SQL generation using logical rewrite rules. Before generating SQL, the SQP may be optimized at the logical level to improve efficiency, correctness, and clarity. Key techniques include filter pushdown, where filters are moved closer to the data source and join reordering, which prioritizes smaller tables first to reduce the size of intermediate results. Thereafter, redundant filter removal, such as simplifying expressions like “ $x = 5 \text{ AND } x > 0$ ” to “ $x = 5$ ” and predicate simplification, for instance, transforming “ $\text{NOT } (x < 5)$ ” into “ $x \geq 5$ ”; Further, the techniques include constant folding, where constant expressions are evaluated early to reduce computational overhead. These optimizations are database-agnostic, meaning they improve the logical plan before SQL generation, resulting in simpler and more predictable SQL queries that are easier for large language models to produce correctly. Additionally, a well-optimized logical plan leads to better performance in the final SQL query execution and helps catch potential logical errors early, enhancing both efficiency and reliability.

Module 9: Multi-Layer SQP Validator

[0157] In an embodiment, a multi-layer SQP validator module (M9) performs four-layer validation. That is, the module is configured to validate SQP against Phase 1 metadata using 4-layer validation before SQL generation. Before proceeding to SQL generation, the SQP must undergo a comprehensive multi-layer validation process to ensure correctness, efficiency, and security. The first layer, schema validation, verifies that all referenced columns and tables exist within the metadata, preventing basic structural errors. The second layer, semantic validation, ensures that aggregations are valid, the query grain is consistent, and any filter values fall within the allowed domain. The third layer, performance validation, evaluates the query for potential inefficiencies, including excessive data scans or the risk of unintended Cartesian products. Finally, the fourth layer, security validation confirms that the query does not access personally identifiable information (PII) without proper authorization and that the executing

user has the necessary permissions, as detailed in Module 9c. The layered validation is critical because it enables early error detection, capturing 89.1% of errors before costly SQL generation, and results in measurable cost savings of approximately \$0.02 per caught error by avoiding unnecessary execution. Additionally, the system provides clear and actionable error messages, such as suggesting correct column names (“Did you mean PAYMENT_VOLUME?”), and prevents catastrophic scenarios by blocking queries that could scan billions of rows or violate security protocols. Overall, this approach maintains high reliability, with a very low false positive rate of just 1.1%, ensuring that almost all valid queries pass while minimizing risks.

Module 9a: Performance Validator and Cost Estimator

[0158] In an embodiment, a performance validator and cost estimator module is configured to predict query performance and prevent expensive queries before execution (Layer 3 validation).

Module 9b: Security Validator Reference and Module 9c: Security Validator & Access Control Enforcer

[0159] In an embodiment, the module is configured to ensure the query does not access PII columns or data user does not have permission to view (Layer 4 validation). The process flow for executing a query begins with extracting the complete list of columns referenced in the SQL statement. Once the columns are identified, the module checks for the presence of any sensitive data, such as personally identifiable information (PII) or payment card information (PCI), associated with the queried columns. Thereafter, the user’s permissions are validated to ensure that their role authorizes access to the requested data. If access to sensitive columns is required, automatic data masking is applied to protect confidential information. Simultaneously, every access attempt is recorded in an audit log to maintain a comprehensive compliance trail. Finally, the module determines whether to block or allow the query, unauthorized requests are rejected, while authorized requests are executed with the necessary safeguards in place. The process ensures adherence to regulatory frameworks such as GDPR, CCPA, and PCI-DSS, protects sensitive data from accidental exposure, maintains a complete audit trail for compliance purposes, and enables safe querying through automated masking mechanisms.

Module 9d: Explanation Generator and User Feedback Interface

[0160] In an embodiment, an explanation generator and user feedback interface module is configured to generate human-readable explanations of how the system interpreted the query and why. For every query, generate an explanation which shows:

1. What You Asked: Parsed natural language
2. How We Understood It: Extracted intent, entities, filters
3. What We're Querying: Tables, columns, joins
4. Why This Interpretation: Confidence scores, evidence
5. Alternative Meanings: Other possible interpretations (if any)

Module 10: Query Success Tracker and Learning Loop

[0161] In an embodiment, a query success tracker and learning loop module is configured to track query execution success/failure, learns from query outcomes and update confidence scores, enabling continuous improvement through feedback-driven adjustments. In Module 10, after every query executes, regardless of whether it succeeds or fails, the module systematically tracks multiple key aspects to ensure continuous improvement. It monitors execution metrics such as latency, cost, and result quality, providing insights into the efficiency and effectiveness of each query. Confidence validation is performed to verify whether high-confidence predictions or responses were actually correct, while user feedback, including thumbs up or down, corrections, and comments, is collected to capture real-world evaluation. The module also conducts A/B testing to compare different strategies, such as template-based approaches versus LLM-generated responses or variations in threshold settings. Based on these inputs, learning actions are undertaken to enhance system performance, which may include updating synonym weights, adjusting confidence thresholds, or adding new templates to the library. The process is critical because it allows the system to self-improve over time, learning from errors and user corrections, while calibrating confidence levels to minimize false positives and negatives. It also facilitates template discovery by identifying new query patterns and optimizes overall performance by determining which strategies are most effective for specific query types.

[0162] In an embodiment, a multi-level semantic cache module implements a three-tier caching strategy: L1 Exact Fingerprint matching using hash and ID; L2 Parametric Template matching for date variations; and L3 Semantic Similarity for finding similar queries. The module achieves 42.3% cache hit rate, which is 8x better than text-based caching approaches.

Phase 2 Outputs

[0163] In an embodiment, Phase 2 produces a validated SQP in JSON format, a confidence score with an average of 0.87, and a cache fingerprint for further reuse. Complexity score (Simple/Medium/Complex) driving model routing; Clarification Request if confidence is below 0.80; Resolved Entities mapping user terms to database columns. Success metrics includes 93.7% intent classification accuracy, 42.3% semantic cache hit rate (8x better than text-based), and 83.2% ambiguity detection precision.

Phase 2 Innovations

[0164] In an embodiment, Phase 2 innovations include SQP as compiler intermediate representation enabling database-agnostic processing, multi-level semantic cache with parametric template matching, context-aware multi-intent detection with parallel execution, confidence-based ambiguity detection with multi-signal scoring, and implicit entity inference from user context achieving 82% accuracy. Also includes fuzzy synonym resolution with multi-hop chain expansion, temporal expression parser with smart granularity detection, SQP template library with 200+ validated templates achieving 34.7% match rate, multi-layer validation across schema, semantic, performance, and security dimensions, query complexity analyzer with ML-based cost prediction, and contextual entity linker with TKG integration. Further, Phase 2 includes SQP optimizer with logical rewrite rules, including filter pushdown, join reordering, and predicate simplification. Furthermore, Phase 2 includes fingerprint-based semantic deduplication using intent-based canonicalization, learning from user corrections with context-aware confidence adjustment and parallel intent execution with smart result merging using union, join, or nested strategies. The input and output of Phase 2 are shown in Tables 8 and 9 below.

Input	Description
Natural Language Query	"Show me payment growth for USA to Canada last month"
Phase 1 Indexes	Hash, Vector, Knowledge Graph for entity resolution
Session Context	Previous queries in same session (for follow-ups like "now show fraud rate")
User Profile	Preferred regions, common metrics (optional)

Table 8

Output	Description
Semantic Query Plan (SQP)	Structured JSON with intent, metrics, filters, time range
Confidence Score	0.0-1.0 indicating how sure we are (threshold: 0.80)
Complexity Score	Simple/Medium/Complex - drives model routing
Clarification Request	If confidence < 0.80: "Did you mean payment volume or transaction count?"
Resolved Entities	"USA" → "UNITED STATES OF AMERICA", "fraud rate" → "CY_FRAUD_RATE"

Table 9

Example SQP Output:

```
{
  "intent": "growth_calculation",
  "metrics": ["CY_PAYMENT_VOLUME", "PY_PAYMENT_VOLUME"],
  "filters": {
    "ISSUER_COUNTRY_NAME": "UNITED STATES OF AMERICA",
    "MERCHANT_COUNTRY_NAME": "CANADA"
  },
  "time_range": {"period": "202411", "comparison": "202311"},
  "aggregation": "SUM",
  "confidence": 0.91
}
```

Phase 2 Significance

[0165] In an embodiment, without Phase 2, the LLM would guess at user intent, there would be no way to validate understanding, no caching would be possible, and ambiguity would cause incorrect SQL generation. The separation of understanding from SQL generation enables cleaner, more debuggable processing, catches ambiguity early, enables effective caching with 42.3% hit rate, maintains database-agnostic processing for portability across DB2, Postgres, and Snowflake, and handles multi-intent queries appropriately (Table 10).

Significance	Impact
Separation of concerns	Understanding $\neq$ SQL generation (cleaner, debuggable)
Catches ambiguity early	Asks clarifying questions BEFORE writing wrong SQL
Enables caching	Same intent = same SQP = cache hit (42.3% hit rate)
Database-agnostic	SQP works for DB2, Postgres, Snowflake - SQL generated later
Multi-intent handling	"Show volume AND fraud rate" $\rightarrow$ detects 2 metrics
Implicit inference	Understands "last month" means "November 2024" automatically

Table 10

Phase 3: Context Driven SQL Synthesis

[0166] In an embodiment, a context driven SQL synthesis module is configured to convert SQP into optimized, constraint-compliant SQL. The average latency is 187ms, representing 71% reduction through template optimization. In Phase 3, the system translates logical intent into executable SQL by applying a structured and constraint-driven approach. It begins by retrieving only the relevant metadata through a hybrid three-layer search combining hash, vector, and graph-based methods. To ensure accuracy and prevent common errors such as hallucinated joins or unintended Cartesian products, hard constraints are programmatically enforced on the SQL generation process. The LLM then generates SQL using a pruned context, reducing the input from 2400 tokens to 432 tokens, which focuses the model on the most essential information. The output is validated SQL that is ready for execution in Phase 4. The key innovation of this phase lies in its constraint-driven generation methodology, which guides the LLM to produce precise SQL patterns rather than relying on unrestricted, blind model predictions.

Module 1: Hybrid Retrieval Engine

[0167] In an embodiment, a hybrid retrieval engine module (M1) performs a 3-layer complementary search strategy combining hash lookup, vector search, and graph traversal to fetch relevant metadata. The hash index provides $O(1)$ exact matching, the vector database

enables semantic similarity for fuzzy cases, and the knowledge graph provides relationship context and valid join paths.

Module 2: Context Pruner and Token Optimizer

[0168] In an embodiment, a context pruner and token optimizer module (M2) configured to reduce metadata context from 2,400 tokens to 432 tokens, achieving 82% reduction with only 0.3% miss rate. Existing Techniques like Relevance-Based Filtering, Progressive Context Loading and Context Compression Techniques (Technique 1: Abbreviate Repeated Patterns, Technique 2: Remove Redundant Metadata) are naïve and rule-based.

[0169] In an embodiment, a novel method for dynamically pruning database schema context for natural language query processing using entropy-weighted semantic relevance scoring with causal dependency graph traversal is disclosed. The Entropy-Weighted Semantic Relevance Pruning (EWSRP) algorithm introduces a structured, multi-stage approach to selecting high-value entities for downstream processing, particularly in SQL generation tasks. In the first stage, Semantic Relevance Decay (SRD), the algorithm calculates relevance scores for entities based on their proximity to query focal points, which are explicitly mentioned entities. The relevance of an entity is computed as an exponentially decaying function of the graph distance to these focal points, augmented by a semantic similarity boost derived from embedding comparisons. Mathematically, the relevance score $R(e)$ of an entity e is expressed

$$R(e) = \sum [w_f \times \lambda^{d(e,f)} \times (1 + \beta \times \text{sim}(e,f))] \dots (1)$$

Where:

- $R(e)$ = Relevance score of entity e
- F = Set of focal points
- w_f = Weight of focal point f
- λ = Decay factor (typically 0.7)
- $d(e,f)$ = Graph distance between e and f
- β = Semantic boost factor (typically 1.5)
- $\text{sim}(e,f)$ = Cosine similarity between embeddings

[0170] In an embodiment, in the second stage, Information Entropy Weighting (IEW) assigns higher significance to metadata with greater information density, ensuring that data contributing the most unique or informative content is emphasized during pruning. The priority score $P(e)$ is computed as

$$P(e) = R(e) \times (1 + \log_2(1 + H(e))) \times U(e) \dots (2)$$

Where:

- $P(e)$ = Priority score of entity e
- $R(e)$ = Relevance score from Stage 1
- $H(e)$ = Information entropy of entity (Shannon entropy)
- $U(e)$ = Uniqueness factor (inverse of redundancy)

[0171] In an embodiment, in the third stage, Causal Dependency Preservation (CDP), EWSRP guarantees that the inclusion of any entity is accompanied by all its necessary causal dependencies, preventing hallucinations or logical gaps during SQL generation. These dependencies include formula-related required columns, join-related foreign key columns, constraint-referenced columns, and aggregation-specific type or cast columns. Further, EWSRP performance metrics and quality metrics are tabulated in Tables 11 and 12.

Query Type	Input Tokens	Output Tokens	Reduction
Simple queries	2,400	180	92.5%
Complex queries	8,500	720	91.5%
Multi-table joins	12,000	1,100	90.8%
Average	7,633	667	91.3%

Table 11

Metric	Value
Context Completeness	99.2%
Missing Dependency Rate	0.3%
LLM Success Rate (with EWSRP)	97.8%
LLM Success Rate (without)	84.2%

Table 12

Module 3: Constraint Injection System

[0172] In an embodiment, a constraint injector module (M3) injects business rules, data types, SQL patterns, and mandatory filters into the LLM prompt. The module is configured to programmatically constrain LLM to prevent hallucinations.

Module 4: LLM SQL Generator

[0173] In an embodiment, an LLM SQL generator module (M4) is configured to generate optimized SQL from SQP along with pruned context and constraints. The module implements a hybrid approach where 78% of queries use validated templates and 22% require LLM generation.

Multi-Model Strategy

Multi-Objective Bayesian Optimization Router (MOBOR) Algorithm

[0174] In an embodiment, a method for dynamic LLM model selection in natural language to SQL systems using multi-objective Bayesian optimization with quality-cost-latency Pareto frontier exploration and contextual bandit-based exploration-exploitation balancing. In an embodiment, the Multi-Objective Bayesian Optimization Router (MOBOR) algorithm is configured to optimize multiple competing objectives during the query routing and SQL generation process. In particular, the MOBOR algorithm simultaneously considers three key performance metrics: quality (Q), cost (C), and latency (L). The quality objective (Q) corresponds to the correctness of the generated SQL query and its execution success rate when applied to the underlying database system. The cost objective (C) represents the computational and monetary expenses incurred during the SQL generation process, including API usage charges and token consumption associated with large language model (LLM) calls. The latency objective (L) represents the time required to generate the SQL query and return a response to the user, thereby reflecting the responsiveness of the system. MOBOR extracts four categories of features for model selection optimization:

Query Complexity Features: table_count, join_count, filter_count, aggregation_count, metric_type, temporal_complexity, dimension_cardinality, semantic_ambiguity_score, constraint_density.

Template Matching Features: best_match_similarity, match_confidence, template_success_rate, template_recency, similar_templates_count, gold_standard_available.

System State Features: budget_remaining_ratio, daily_budget_pace, current_load, time_of_day_factor, recent_error_rate, model_availability.

Temporal Pattern Features: hour_of_day, day_of_week, days_until_month_end, is_business_hours, query_velocity_1h.

Bayesian Surrogate Models

[0175] In an embodiment, Gaussian process (GP) regression models are used for predicting quality, cost, and latency for each candidate model. The system maintains separate Gaussian Process (GP) models corresponding to each candidate model used for query generation and execution. These candidate models may include a budget-friendly and fast model such as *gpt-4o-mini*, a balanced model such as *gpt-4-turbo*, a high-quality but relatively expensive model such as *claude-3.5-sonnet*, and a predefined template-based approach that provides near-instant execution with negligible computational cost.

Pareto Frontier Selection

[0176] In an embodiment, a Pareto Frontier selection mechanism represents a core innovation based on multi-objective optimization for identifying non-dominated model configurations. In this approach, multiple objectives such as quality (Q), cost (C), and latency (L) are evaluated simultaneously. A model configuration m_1 is considered to dominate another configuration m_2 when m_1 performs better than or equal to m_2 across all objectives, and demonstrates strictly superior performance in at least one objective. Configurations that are not dominated by any other are retained on the Pareto Frontier, thereby representing optimal trade-off solutions among the competing objectives.

Adaptive Scalarization

[0177] In an embodiment, an adaptive scalarization is implemented through dynamic adjustment of optimization weights based on the current system state and operational constraints. In this approach, the relative importance of cost, quality, and latency is continuously modified to reflect real-time conditions. For example, when the remaining budget falls below a critical threshold of 10%, the system prioritizes cost efficiency by assigning a cost weight of 0.6, while quality and latency are assigned weights of 0.3 and 0.1, respectively. In situations where projected spending indicates a potential budget overshoot, the weights are adjusted to cost 0.45, quality 0.4, and latency 0.15 to balance expenditure control with service quality. Conversely, when the system is underspending near the end of the budget cycle, the emphasis shifts toward improving output performance, with weights of 0.15 for cost, 0.6 for quality, and 0.25 for latency. Additionally, if there is a risk of violating a latency service-level agreement (SLA), the latency weight is increased by 0.15 to ensure timely responses.

Thompson Sampling for Exploration

[0178] In an embodiment, a thompson Sampling for exploration is implemented to balance the exploitation of the best-performing known model with the exploration of alternative configurations that may yield improved results.

$$\varepsilon(t) = \varepsilon_0 \times \gamma^t \dots (3)$$

Where exploration rate decays over time as the system learns optimal routing. Further, the MOBOR performance metrics and model utilization breakdown are tabulated in Tables A and B.

Metric	Value	Improvement
Cost savings vs static routing	34.2%	\$847.50/month
SQL success rate	97.8%	+6.3% vs baseline
P50 latency	185ms	-23% vs baseline
Template utilization	62.4%	Near-zero cost queries

Table A

Model	Usage %
Template	62.4%
gpt-4o-mini	24.1%
gpt-4-turbo	10.8%
claude-3.5-sonnet	2.7%

Table B

Module 5: SQL Syntax Validator

[0179] In an embodiment, a SQL syntax validator (M5) module parses the generated SQL and checks schema compliance before execution, that is, fast, deterministic validation before execution. Validation includes syntax correctness, table and column existence, data type compatibility, and constraint adherence.

Module 6: SQL Generation Optimizer and Learning Loop

[0180] In an embodiment, a SQL generation optimizer and learning loop (M6) module tracks constraint effectiveness, performs ML-based performance prediction with an MAE of 0.19 seconds, and automatically extracts successful patterns into the template library. The A/B testing framework enables real-time strategy optimization. In other words, the module is configured to self-improving SQL generation with cost optimization, template reuse, and

adaptive constraint tuning. Integration with Phase 4 occurs through a feedback-driven promotion mechanism in which templates extracted in this phase are elevated to a “Gold Standard Library” once users verify the correctness of results during Phase 4 (Component 6.5). When users provide positive feedback, such as a “Thumbs Up” or explicit verification, the system updates internal rankings to increase the priority of those templates.

[0181] In an embodiment, Phase 3 transforms a validated semantic query plan (SQP) into production-ready SQL through a hybrid pipeline that balances retrieval speed, constraint enforcement, and AI-driven SQL generation. By injecting metadata-derived constraints directly into the LLM prompt, this phase guarantees schema correctness and business rule compliance while minimizing hallucination risk.

[0182] In an embodiment, the cost-aware model routing (MOBOR algorithm) module implements dynamic model selection based on query complexity and budget constraints. Simple queries route to cost-effective models (GPT-4o-mini at \$0.002), medium complexity to balanced models (Claude 3.5 at \$0.015), and high complexity to premium models (GPT-4 Turbo at \$0.008). The algorithm uses Bayesian surrogate models, Pareto frontier computation, and Thompson Sampling to optimize the cost-quality tradeoff, achieving 68% cost reduction while maintaining accuracy.

[0183] **FIGs. 10 and 11** illustrate a flow diagram for a constrained Structured Query Language (SQL) generation pipeline (Phase 3) showing the processing pipeline from Valid SQP input through to Valid SQL output, template matching and cost-aware routing mechanisms, in accordance with some embodiments of the present disclosure. In an embodiment, as illustrated in **FIG. 10**, the sequential flow diagram shows external stores containing Phase 1 metadata and a learning store with templates, costs, and performance data. The sequential flow proceeds from valid SQP from Phase 2 through M1 hybrid retrieval (Hash + Vector + Graph), M2 context pruner for filtering relevant metadata, M3 constraint injector for adding business rules and SQL patterns, M4 SQL generator for LLM or template generation, M5 syntax validator for parsing and validating SQL, and M6 optimizer and learning for cost routing, performance prediction, and template updates, ultimately producing valid SQL to Phase 4. In an embodiment, as shown in **FIG. 11**, the detailed flowchart depicts valid SQP input flowing to M0 check template library decision point, branching based on match score to template instantiation (for high match), template and LLM refinement (for medium match), or M1 hybrid retrieval (for no match). The

hybrid retrieval branches into three parallel layers: Layer 1 Hash Lookup, Layer 2 Vector Search, and Layer 3 Graph Traversal, with results merged and processed through M2 context pruner, self-correction loop, and M3 constraint injector. The M7 cost-aware model router directs queries to appropriate LLM models (GPT-4o-mini, Claude 3.5, GPT-4 Turbo) based on budget and complexity. Phase 3 learning store contains a template library with 1,847 patterns, a cost tracker with budget information, and performance Logs with ML model data.

Phase 3 Outputs

[0184] In an embodiment, the phase 3 produces production-ready SQL with 99.8% constraint compliance, estimated execution time with MAE of 0.19 seconds and performance optimization suggestions.

Phase 3 Success Metrics

[0185] In an embodiment, the phase 3 achieves 94% SQL generation success rate, 0.3% schema hallucination rate representing 86% reduction from baseline, \$137/month LLM cost representing 68% reduction through optimization, and 78% template match rate.

Phase 3 Innovations

[0186] In an embodiment, Phase 3 key innovations include Automatic SQL template extraction with 1,847 self-learned patterns, cost-aware multi-model routing using MOBOR Algorithm with Bayesian optimization, constraint effectiveness tracking with feedback-driven tuning achieving 0.92 average effectiveness, ML-based performance prediction with 89% accuracy within 1 second, A/B testing framework for real-time SQL generation strategy optimization, semantic context pruning using EWSRP algorithm achieving 82% token reduction, hybrid template-LLM refinement strategy for medium-match queries, and cross-phase feedback loop integration enabling continuous improvement. The input and output of Phase 3 are shown in Tables 13 and 14 below.

Input	Description
Validated SQP	From Phase 2 - structured query plan
Phase 1 Metadata	Only relevant columns/tables (pruned context)
Constraint Rules	Mandatory CASTs, allowed joins, forbidden patterns
SQL Template Library	1,847 proven patterns from past successful queries

Input	Description
Cost Budget	Monthly LLM spend limit for routing decisions

Table 13

Output	Description
Production SQL	Ready-to-execute, optimized query
Estimated Execution Time	ML prediction: "~2.3 seconds"
Cost Tracking	"\$0.002 for this query, \$3,247 monthly spend"
Validation Status	Syntax ✓, Schema ✓, Constraints ✓
Optimization Hints	"Consider adding index on MONTH_YEAR"

Table 14

Phase 3 Significance

[0187] In an embodiment, without Phase 3, the full schema would be sent to the LLM (expensive), no constraints would prevent hallucinations, no templates would enable reuse (slow), and no cost control would manage budget. The 82% token reduction provides massive cost savings, template reuse eliminates LLM calls for 78% of queries, constraint injection prevents forbidden operations, and cost-aware routing optimizes the quality-cost tradeoff (Table 15).

Significance	Impact
82% token reduction	2,400 → 432 tokens = massive cost savings (\$0.003 vs \$0.08)
Template reuse	78% of queries use proven templates - no LLM needed
Constraint injection	LLM CAN'T make forbidden joins or skip CASTs
No hallucinations	0.3% hallucination rate (vs 23% industry standard)
Cost-aware routing	Simple queries → cheap model, complex → premium
Pre-validation	Catches errors BEFORE hitting database

Table 15

Phase 4: Guardian Loop – Validation and Self-Healing

[0188] In an embodiment, the guardian loop module is configured to execute, validate, self-heal, learn, and provide intelligent insights. The average overhead is 43ms, excluding database execution time. Input for phase 4 is from the generated SQL from Phase 3. Phase 4 functions as a quality assurance and learning layer that executes the generated SQL queries within the database environment and processes the returned results through deterministic parsing without relying on a language model for validation. If query execution fails, the system performs a one-shot self-correction and automatically repairs common issues such as type casting errors, NULL handling problems, or missing LIMIT clauses. The outcomes of both successful and failed executions are fed back into Phase 1 metadata, enabling updates to trust scores, query patterns, and synonym mappings. In addition, successfully executed queries are cached for rapid retrieval in future requests. Through this self-healing loop, each execution outcome continuously improves the system, allowing errors to refine corrective logic and successes to strengthen the knowledge base.

Component 1: Deterministic SQL Parser

[0189] In an embodiment, a SQL executor (C1) module is configured to execute the validated SQL against the target database with instrumentation for performance monitoring, capturing execution time, row counts, and resource utilization. The main purpose of the module is non-LLM validation for syntax, schema correctness and constraint compliance.

Component 2: SQL Execution Engine

[0190] In an embodiment, a SQL execution engine is configured to execute validated SQL and capture detailed metrics. In an embodiment, a result parser and validator module (C2) implements 4-layer validation:

- (a) Layer 1 - Syntax Validation: Verifies SQL executed without errors.
- (b) Layer 2 - Schema Validation: Confirms returned columns match expected schema.
- (c) Layer 3 - Business Logic Validation: Checks results against business rules such as "fraud rate must be 0-100%" and "volume cannot be negative."
- (d) Layer 4 - Semantic Validation: Verifies results are semantically consistent with the original query intent.

Component 3: Self-Correction Engine

[0191] In an embodiment, when errors occur, a self-corrector module (C3) performs root cause analysis and automatic SQL rewrite, achieving 78% success rate without manual intervention. More specifically, the module is configured to one-shot automatic fix attempt for common errors. The system analyzes error messages, identifies the likely cause, and generates corrected SQL.

Component 4: Auto-Repair System

[0192] In an embodiment, an auto-repair engine module (C4) maintains a library of 234 error patterns with corresponding repair strategies, achieving 99.6% repair success rate for known patterns. More specifically, the module is configured to fix common patterns automatically (no LLM needed). Repairs are applied proactively based on pattern matching.

Component 5: Execution Monitor & Metrics Capture

[0193] In an embodiment, a performance monitor module (C5) is configured to track latency, cost, and quality metrics for each query, feeding data into the ML-based performance prediction models and cost optimization algorithms. More specifically, the module is configured to capture comprehensive metrics for learning.

Component 6: Learning Feedback Loop

[0194] In an embodiment, a learning feedback loop module (C6) is configured to updates Phase 1 metadata, constraints, and templates based on query execution results. Trust scores are propagated through multi-hop relationships with 97.8% accuracy. Successful patterns are promoted to the Gold Standard library with 23.4% promotion rate, and negative rules are automatically generated from failures with 91.3% effectiveness.

Component 7: Result Caching

[0195] In an embodiment, a result caching module (C7) stores results with intent-based fingerprints, achieving 42.3% hit rate through 3-tier caching (exact, parametric, semantic). More specifically, the module is configured to cache successful queries for instant future retrieval.

Component 8: Result Intelligence and Quality Assurance

[0196] In an embodiment, a result intelligence and quality assurance is configured to validate result quality, detect anomalies, provide optimization advice, and track session context for intelligent recommendations. The module provides intelligent insights, including

- (a) Anomaly Detection: Statistical analysis identifies unusual results with 89.2% detection rate and 3.8% false positive rate. Alerts such as "Growth of 450% is unusual - verify data" help users identify potential data quality issues.
- (b) Performance Optimization: Suggestions for query improvement achieve 67% average performance improvement.
- (c) Session Context: The system maintains context across queries in a session, achieving 94.1% accuracy on follow-up queries like "now show fraud rate" after a volume query.
- (d) Drill-Down Suggestions: Contextually relevant follow-up queries are suggested with 86.7% relevance rate, such as "Want to see this by month?" or "Compare with other corridors?".
- (e) Natural Language Explanations: Results are explained in plain language with 4.6/5.0 user satisfaction rating.

[0197] In an embodiment, Human-in-the-Loop (HITL) Gold Standard Library: User feedback including thumbs up/down ratings and corrections is captured and used to promote successful query-SQL pairs to the Gold Standard library. These become few-shot examples for LLM prompts, with rank-based selection ensuring the highest-quality examples are used.

Phase 4 Outputs

[0198] In an embodiment, the phase 3 produces query results with 96.2% final success rate (94% first attempt plus 78% self-correction), anomaly alerts with 89.2% detection rate, optimization recommendations with 67% average improvement, drill-down suggestions with 86.7% relevance, natural language explanations with 4.6/5.0 satisfaction and learning updates including trust scores, new synonyms, and negative rules.

Phase 4 Innovations

[0199] In an embodiment, the phase 4 key innovations include multi-layer result validation framework achieving 96.2% success, intelligent self-correction with root cause analysis at 78% success rate, pattern-based auto-repair with 234 error patterns and 99.6% repair success, trust score propagation with multi-signal confidence at 97.8% accuracy, semantic result caching with intent-based deduplication at 42.3% hit rate, HITL gold standard library with 23.4% promotion rate and rank-based few-shot selection, LLM-powered negative rule generation with 91.3% effectiveness, adaptive index enrichment with feedback-driven synonym updates achieving 8.2% accuracy improvement, statistical anomaly detection with business rule

validation at 89.2% detection and 3.8% false positive rate, and session-aware query intelligence with 94.1% follow-up accuracy and 86.7% suggestion relevance. The input and output of Phase 3 are shown in Tables 16 and 17 below.

Input	Description
Generated SQL	From Phase 3
Database Connection	Execution environment
Historical Patterns	Expected value ranges, typical row counts
Business Rules	"Fraud rate must be 0-100%", "Volume can't be negative"
User Feedback	👍 Thumbs up, 👎 Thumbs down, ✎ Corrections

Table 16

Output	Description
Query Results	Actual data rows
Validation Status	Pass/Fail with reasons
Anomaly Alerts	"Growth of 450% is unusual - verify data"
Self-Correction	If error: auto-fixed SQL + retry
Drill-Down Suggestions	"Want to see this by month?" "Compare with other corridors?"
Natural Language Explanation	"USA to Canada payments grew 8.7% in November 2024"
Learning Updates	Trust scores, new synonyms, negative rules

Table 17

Example Learning Output

```
{
  "trust_updates": {
    "USA → UNITED STATES OF AMERICA": 0.78 → 0.85
  },
  "new_negative_rule": {
    "description": "Never join TRANSACTIONS and MERCHANTS without common key",
```

```
"triggered_by": "Cartesian product error"
},
"template_promoted": {
  "pattern": "corridor_payment_growth",
  "success_rate": "99.2%",
  "status": "Gold Standard"
}
}
```

Phase 4 Significance

[0200] In an embodiment, without Phase 4, there would be no validation (silent failures), no learning (same mistakes repeated), no self-correction (manual debugging required), and no context (every query isolated). The Guardian Loop ensures quality assurance by catching wrong results before showing to users, enables self-healing with 78% of errors auto-fixed transparently, drives continuous learning where the system improves from every query, propagates trust to reinforce good mappings and demote bad ones, builds the Gold Standard library for few-shot examples, learns negative rules to prevent repeat mistakes, maintains session intelligence for contextual follow-ups, and detects anomalies to flag suspicious results (Table 18).

Significance	Impact
Quality assurance	Catches wrong results BEFORE showing to user
Self-healing	78% of errors auto-fixed without user knowing
Continuous learning	System improves from every single query
Trust propagation	Good mappings get reinforced, bad ones get demoted
Gold Standard library	Best patterns become few-shot examples for LLM
Negative rules	Learns what NOT to do (prevents repeat mistakes)
Session intelligence	"Show me fraud rate" after volume query → knows context
Anomaly detection	Flags suspicious results (89.2% accuracy)

Table 18

[0201] Continuous Learning and Feedback Mechanisms

A critical aspect of the ATLAS system is the continuous learning architecture that enables improvement over time. Phase 4 updates Phases 1, 2, and 3 through multiple feedback loops:

Trust Score Propagation: When queries succeed or fail, trust scores for entity mappings, synonyms, and templates are updated. Successful mappings like "USA" → "UNITED STATES OF AMERICA" see confidence increase from, for example, 0.45 to 0.85, while failed mappings are demoted from, for example, 0.78 to 0.65.

Template Promotion: SQL patterns that achieve high success rates (e.g., 99.2%) are promoted to Gold Standard status and become primary templates for similar future queries.

Negative Rule Generation: When errors occur, the system uses LLM analysis to generate negative rules such as "Never join dimensional tables without fact table" preventing the same mistake from recurring.

Synonym Updates: New synonyms discovered through user queries are added to the hash index with appropriate confidence scores, expanding the system's vocabulary over time.

Constraint Refinement: Constraint effectiveness is tracked, and constraints that consistently prevent errors are strengthened while ineffective constraints are refined or removed.

End-to-End Example

[0202] In an embodiment, the complete system operation considers the following example query: "Show USA to Canada payment volume growth for September 2024." As shown in **FIG. 12**, the NLQ to SQL conversion process demonstrates the multi-tenant architecture with inverted index O(1) lookup combined with a large language model approach. The flowchart shows an input query example proceeding through an application router, table selection, context builder performing O(1) hash lookup and metadata enrichment, few-shot examples retrieval, context package assembly, LLM SQL generator, and SQL output generation, demonstrating the O(1) hash-based lookup innovation, achieving 0.004 second context building time.

[0203] In an embodiment, in the metadata factory (Phase 1) (Pre-built, 0ms) module, the indexes are already built and ready, including a hash index with 45,678 entries, a vector DB with 12,345 embeddings, a knowledge graph with 8,234 entities and 15,678 relationships, and a constraint store with 456 business rules.

[0204] In an embodiment, in the query understanding (Phase 2 – 68ms) module, intent Classification (8ms) identifies "growth_calculation" as primary intent with 0.92 confidence. Ambiguity detection (12ms) scores ambiguity at 0.15 with 0.85 confidence, proceeding without clarification. Entity extraction (25ms) identifies metrics [CY_PAYMENT_VOLUME,

PY_PAYMENT_VOLUME], source geography "UNITED STATES OF AMERICA", destination geography "CANADA", and temporal period 202409 with YoY comparison. Semantic cache check (5ms) finds no exact match but 3 similar queries. Further, SQP generation (18ms) creates the structured query plan with 0.87 confidence and 0.42 complexity score. The detailed steps are explained as follows:

Step 1: Intent Classification (8ms)

```
{
  "primary_intent": "growth_calculation",
  "sub_intent": "metric_retrieval",
  "query_type": "analytical",
  "confidence": 0.92
}
```

Step 2: Ambiguity Detection (12ms)

```
{
  "ambiguity_score": 0.15,
  "confidence": 0.85,
  "action": "proceed",
  "reason": "High confidence, no clarification needed"
}
```

Step 3: Entity Extraction (25ms)

```
{
  "entities": {
    "metrics": ["CY_PAYMENT_VOLUME", "PY_PAYMENT_VOLUME"],
    "source_geography": "UNITED STATES OF AMERICA",
    "destination_geography": "CANADA",
    "temporal": {
      "period": 202409,
      "grain": "MONTHLY",
      "comparison": "YoY"
    },
    "modifier": "growth"
  }
}
```

Step 4: Semantic Cache Check (5ms)

```
{
  "cache_hit": false,
  "similar_queries": 3,
  "action": "generate_sqp"
}
```

Step 5: SQP Generation (18ms)

```
{
  "sqp": {
    "intent": "growth_calculation",
    "base_metrics": ["CY_PAYMENT_VOLUME",
    "PY_PAYMENT_VOLUME"],
    "derived_metric": {
      "name": "growth_pct",
      "formula": "100 * (CY - PY) / PY"
    },
    "filters": [
      {"column": "ISSUER_COUNTRY_NAME", "operator": "=", "value":
      "UNITED STATES OF AMERICA"},
      {"column": "MERCHANT_COUNTRY_NAME", "operator": "=", "value":
      "CANADA"},
      {"column": "MONTH_YEAR", "operator": "=", "value": 202409}
    ],
    "aggregation": "SUM",
    "grain": "corridor_monthly"
  },
  "confidence": 0.87,
  "complexity": 0.42
}
```

Phase 2 Output: Valid SQP with 0.87 confidence

[0205] In an embodiment, in SQL synthesis (Phase 3 - 182ms) module, template check (15ms) finds 0.92 match score with "corridor_growth_calculation" template. Hybrid retrieval (45ms) identifies the relevant table and 5 columns. Context pruning (12ms) reduces from 2,400 to 432

tokens (82% reduction). Template instantiation (25ms) generates the SQL with proper CAST operations, SUM aggregations, and growth percentage calculation with null handling. Syntax validation (25ms) confirms syntax valid, schema valid, and no constraint violations. Further, performance prediction (35ms) estimates a 2.1 second execution time and cost tracking (25ms) records \$0.0001 cost using the template strategy. The detailed steps are explained as follows:

Step 1: Template Check (15ms)

```
{
  "template_match": 0.92,
  "template_id": "corridor_growth_calculation",
  "action": "use_template",
  "reason": "High match score, template covers all requirements"
}
```

Step 2: Hybrid Retrieval (45ms)

```
{
  "tables": ["TBL_XB_KPIS_PV_TXN_AGG_GLOBAL_TBLUE_FEED"],
  "columns": ["CY_PAYMENT_VOLUME", "PY_PAYMENT_VOLUME",
    "ISSUER_COUNTRY_NAME",
    "MERCHANT_COUNTRY_NAME", "MONTH_YEAR"],
  "relationships": [],
  "constraints": 5
}
```

Step 3: Context Pruning (12ms)

```
{
  "original_tokens": 2400,
  "pruned_tokens": 432,
  "reduction": "82%",
  "included": ["5 columns", "1 formula", "3 constraints"]
}
```

Step 4: Template Instantiation (25ms)

```
SELECT
  SUM(CAST(CY_PAYMENT_VOLUME AS DECIMAL(31,0))) AS
  cy_volume,
```

```

SUM(CAST(PY_PAYMENT_VOLUME AS DECIMAL(31,0))) AS
py_volume,
CASE
WHEN SUM(CAST(PY_PAYMENT_VOLUME AS DECIMAL(31,0))) = 0
THEN NULL
ELSE 100.0 * (
SUM(CAST(CY_PAYMENT_VOLUME AS DECIMAL(31,0))) -
SUM(CAST(PY_PAYMENT_VOLUME AS DECIMAL(31,0)))
) / SUM(CAST(PY_PAYMENT_VOLUME AS DECIMAL(31,0)))
END AS growth_pct
FROM TBL_XB_KPIS_PV_TXN_AGG_GLOBAL_TBLUE_FEED
WHERE ISSUER_COUNTRY_NAME = 'UNITED STATES OF AMERICA'
AND MERCHANT_COUNTRY_NAME = 'CANADA'
AND MONTH_YEAR = 202409;

```

Step 5: Syntax Validation (25ms)

```

{
  "syntax_valid": true,
  "schema_valid": true,
  "constraint_violations": [],
  "status": "READY_FOR_EXECUTION"
}

```

Step 6: Performance Prediction (35ms)

```

{
  "estimated_time": "2.1s",
  "estimated_cost": "$0.003",
  "index_coverage": 0.8,
  "optimization_suggestions": [],
  "action": "execute"
}

```

Step 7: Cost Tracking (25ms)

```

{
  "strategy": "template",
  "cost": "$0.0001",
  "latency": "25ms",

```

```
"budget_remaining": "$3,247 (65%)"
}
```

Phase 3 Output: Optimized SQL ready for execution

[0206] In an embodiment, in the guardian loop (Phase 4 - 2,185ms total) module, SQL execution (2,100ms database time) returns 1 row with cy_volume, py_volume, and growth_pct columns. Result parsing (8ms) extracts values: cy_volume=1,234,567,890, py_volume=1,098,765,432, growth_pct=12.36. Multi-layer validation (18ms) passes all four layers. Anomaly detection (15ms) finds no anomalies with a z-score of 1.2 below the threshold value 3.0. Performance analysis (12ms) confirms prediction accuracy and session context update (8ms) tracks entities for follow-up queries and generates drill-down suggestions. Learning feedback (12ms) updates trust scores and template success rate. Natural language explanation (7ms) generates a summary and a cache result (5ms) stores with a 24-hour TTL. The detailed steps are explained as follows:

Step 1: SQL Execution (2,100ms - database time)

```
{
  "rows_returned": 1,
  "columns": ["cy_volume", "py_volume", "growth_pct"],
  "execution_time": "2.1s",
  "status": "SUCCESS"
}
```

Step 2: Result Parsing (8ms)

```
{
  "parsed_result": {
    "cy_volume": 1234567890,
    "py_volume": 1098765432,
    "growth_pct": 12.36
  },
  "data_type_valid": true,
  "null_handling": "correct"
}
```

Step 3: Multi-Layer Validation (18ms)

```
{
```



```

"layer1_syntax": "PASS",
"layer2_schema": "PASS",
"layer3_business_logic": "PASS",
"layer4_semantic": "PASS",
"overall": "VALID"
}

```

Step 4: Anomaly Detection (15ms)

```

{
  "anomalies_detected": 0,
  "statistical_analysis": {
    "z_score": 1.2,
    "threshold": 3.0,
    "status": "normal"
  },
  "business_rules": {
    "non_negative": "PASS",
    "reasonable_growth": "PASS"
  }
}

```

Step 5: Performance Analysis (12ms)

```

{
  "actual_time": "2.1s",
  "predicted_time": "2.1s",
  "prediction_error": "0.0s",
  "optimization_suggestions": [],
  "performance_grade": "A"
}

```

Step 6: Session Context Update (8ms)

```

{
  "session_pattern": "corridor_analysis",
  "entities_tracked": ["USA", "Canada", "payment_volume"],
  "drill_down_suggestions": [
    "Show fraud rate for USA to Canada",
    "Compare USA→Canada vs USA→UK growth",

```

```

    "Show monthly trend for USA→Canada"
  ]
}

```

Step 7: Learning Feedback (12ms)

```

{
  "trust_updates": [
    {"entity": "USA", "column": "ISSUER_COUNTRY_NAME", "change":
+0.05},
    {"synonym": "US→USA", "confidence": +0.03}
  ],
  "template_success": {
    "template_id": "corridor_growth_calculation",
    "success_rate": "0.998 → 0.999"
  },
  "gold_standard_eligible": true
}

```

Step 8: Natural Language Explanation (7ms)

```

{
  "summary": "Payment volume from USA to Canada in September 2024 was
$1.23B, representing a 12.36% increase compared to September 2023
($1.10B).",
  "key_insights": [
    "Strong YoY growth in USA→Canada corridor",
    "Growth rate above 6-month average (8.7%)",
    "No anomalies detected"
  ],
  "confidence": 0.94
}

```

Step 9: Cache Result (5ms)

```

{
  "cached": true,
  "fingerprint": "sha256:a3f5c8d2e4f6...",
  "ttl": "24h",
  "parametric_template": true
}

```

}

Phase 4 Output: Results + Insights + Suggestions

[0207] In an embodiment, final response to user (2,435ms total), the system 103 returns the answer showing \$1.23B current year volume, \$1.10B prior year volume, and 12.36% growth, along with explanation, insights noting strong YoY growth above 6-month average, drill-down suggestions, and metadata including latency, cost, and confidence score. The detailed steps are explained as follows:

```
{
  "answer": {
    "cy_volume": "$1.23B",
    "py_volume": "$1.10B",
    "growth_pct": "12.36%"
  },
  "explanation": "Payment volume from USA to Canada in September 2024 was $1.23B,
representing a 12.36% increase compared to September 2023 ($1.10B). This growth
rate is above the 6-month average of 8.7%, indicating strong performance in this
corridor.",
  "insights": [
    "✅ Strong YoY growth in USA→Canada corridor",
    "📊 Growth rate (12.36%) above 6-month average (8.7%)",
    "🔍 No data anomalies detected"
  ],
  "drill_down_suggestions": [
    "Show fraud rate for USA to Canada Sep 2024",
    "Compare USA→Canada vs USA→UK growth Sep 2024",
    "Show USA→Canada monthly trend for 2024"
  ],
  "metadata": {
    "latency": "2.4s",
    "cost": "$0.0034",
    "confidence": 0.94,
    "cache_enabled": true
  }
}
```

```

    }
  }

```

Complete Innovation Summary: Major Unique Points coming together in an end-to-end solution

[0208] In an embodiment, in Phase 1, the metadata factory module inverts the index with multi-level synonym expansion and Temporal Knowledge Graph (TKG) with schema versioning.

[0209] Phase 2: Query Understanding:

1. Semantic Query Plan (SQP) as Intermediate Representation - Database-agnostic compiler IR
2. Multi-Level Semantic Cache with Parametric Template Matching - 42.3% hit rate (8x better)
3. Context-Aware Multi-Intent Detection with Parallel Execution - Handles compound questions
4. Confidence-Based Ambiguity Detection - Multi-signal confidence scoring (0.80 threshold)
5. Implicit Entity Inference from User Context - Infers missing entities (82% accuracy)
6. Fuzzy Synonym Resolution with Multi-Hop Chain Expansion - Confidence propagation
7. Temporal Expression Parser with Smart Granularity Detection - Auto-detects grain (daily/monthly/quarterly)
8. SQP Template Library with Smart Pattern Matching - 200+ validated templates (34.7% match)
9. Multi-Layer SQP Validation - Schema + Semantic + Performance + Security (89.1% pass rate)
10. Query Complexity Analyzer with Cost Prediction - ML-based prediction (50K training samples)
11. Contextual Entity Linker with TKG Integration - 87% probability scoring
12. SQP Optimizer with Logical Rewrite Rules - Filter pushdown, join reordering, predicate simplification
13. Fingerprint-Based Semantic Deduplication - Intent-based, not text-based canonicalization
14. Learning from User Corrections - Context-aware confidence adjustment.

15. Parallel Intent Execution with Smart Result Merging - Union vs join vs nested strategies

[0210] Phase 3: SQL Synthesis

1. Hybrid SQL Template Library with Automatic Pattern Extraction - 1,847 self-learned patterns (99.2% success)
2. Cost-Aware Model Router with Dynamic Budget Management - 3 LLM tiers, \$5K budget tracking (65% utilization) – EWSRP Algorithm
3. Constraint Effectiveness Tracker - Feedback-driven tuning (0.92 avg effectiveness)
4. ML-Based Performance Prediction Model - MAE: 0.19s (89% accuracy within 1s)
5. A/B Testing Framework for SQL Generation - Real-time strategy optimization (78% template, 22% LLM)
6. Semantic Context Pruning with Relevance Scoring - 82% token reduction (0.3% miss rate)
7. Hybrid Template-LLM Refinement Strategy - 75-90% match → template skeleton + LLM tweaks (99.1% success) – MOBOR Algorithm
8. Feedback Loop Integration with Phase 2 & Phase 4 - Cross-phase learning propagation

[0211] Phase 4: Guardian Loop

1. Multi-Layer Result Validation Framework - Syntax + Schema + Business Logic + Semantic (96.2% success)
2. Intelligent Self-Correction with Root Cause Analysis - Automatic SQL rewrite (78% success rate)
3. Pattern-Based Auto-Repair with Historical Learning - 234 error patterns, 99.6% repair success
4. Trust Score Propagation with Multi-Signal Confidence - Multi-hop propagation (97.8% accuracy)
5. Semantic Result Caching with Intent-Based Deduplication - 3-tier cache (42.3% hit rate)
6. Human-in-the-Loop (HITL) Gold Standard Library - 23.4% promotion rate, rank-based Few-Shot selection
7. LLM-Powered Negative Rule Generation - Automatic constraint learning (91.3% effectiveness)
8. Adaptive Index Enrichment with Feedback-Driven Synonym Updates - 8.2% accuracy improvement over 3 months

9. Statistical Anomaly Detection with Business Rule Validation - 89.2% detection, 3.8% false positive
10. Session-Aware Query Intelligence with Multi-Query Context Tracking - 94.1% follow-up accuracy, 86.7% suggestion relevance.

[0212] In an embodiment, the disclosed system provides measurable business and technical impact, which may be evaluated using defined performance metrics, cost efficiency indicators, and continuous improvement measures are tabulated in Tables 19-21.

Metric	Industry Baseline	ATLAS Solution	Improvement
Success Rate	76-82%	96.2%	+14-20 points
End-to-End Latency	950ms	305ms	68% faster
Cache Hit Rate	5-10% (text)	42.3% (semantic)	8.5x better
Schema Hallucination	2-5%	0.3%	86-94% reduction
Self-Correction Rate	0% (manual)	78% (auto)	New capability
Anomaly Detection	0% (manual)	89.2% (auto)	New capability

Table 19

Category	Annual Savings	Source
LLM Cost Optimization	\$8,732	Template usage (78%), cost-aware routing
Error Prevention	\$16,801	Self-correction (78%), auto-repair (234 patterns)
Cache Efficiency	\$2,888	42.3% cache hit rate
Anomaly Prevention	\$105,233	89.2% detection, 623 bad decisions prevented
Performance Optimization	\$27,315	67% avg query time reduction
Manual Effort Reduction	\$10,657	78% self-correction eliminates manual fixes

Total Annual Savings	\$171,626	Across 341,436 queries/year
-----------------------------	-----------	-----------------------------

Table 20

Metric	Month 1	Month 3	Growth
SQL Template Library	200 patterns	1,847 patterns	9.2x
Gold Standard Library	200 queries	6,667 queries	33x
Error Pattern Library	50 patterns	234 patterns	4.7x
Entity Linking Accuracy	87%	92%	+5 points
Constraint Effectiveness	0.82 avg	0.91 avg	+11%
SQL Generation Accuracy	89%	94%	+5 points

Table 21

[0213] In an embodiment, the disclosed system provides technical advantages over traditional text-to-SQL systems by employing a multi-phase processing architecture that introduces structured intermediate representations, validation layers, and optimization logic between natural language input and query execution (as shown in Table 22).

Feature	Traditional Systems	ATLAS Solution	Advantage
Architecture	Direct NL → SQL	4-phase pipeline with SQP IR	Validation + optimization layers
Success Rate	76-82%	96.2%	+14-20 points
Caching	Text-based (5% hit)	Semantic intent (42.3% hit)	8.5x better
Error Handling	Manual fixes	78% self-correction	Automated recovery
Learning	Static	Continuous (3 feedback loops)	Self-improving
Cost Optimization	None	68% reduction through routing	\$8,732/year savings
Anomaly Detection	None	89.2% coverage	Prevents bad decisions
Session Intelligence	None	94.1% follow-up accuracy	Multi-query awareness

User	None	HITL Gold	User-driven quality
Feedback		Standard (23.4%)	

Table 22

[0214] In an embodiment, the system is distinguished by a database-agnostic structured query plan intermediate representation that enables semantic abstraction across heterogeneous data sources, coupled with self-healing query generation capabilities that automatically correct a substantial portion of detected errors using learned error patterns. The system incorporates cost-aware intelligence for dynamic model routing and budget management, entropy- and Bayesian-based optimization algorithms for decision weighting and trade-off optimization, and a multi-layer validation framework that intercepts erroneous queries early in the generation process. Additional differentiators include human-in-the-loop feedback mechanisms that drive few-shot learning, cross-phase learning loops that propagate improvements across all processing stages, and result-level intelligence configured to detect anomalies and generate insights and explanations, thereby enabling a production-ready architecture optimized for accuracy, performance, and operational efficiency.

[0215] In an embodiment, the ATLAS agent represents a paradigm shift in natural language database querying by implementing a 4-phase pipeline with continuous learning loops. As a result, the system achieves 96.2% success rate (industry-leading accuracy), 305ms end-to-end latency (68% faster than baseline), \$171K annual savings through intelligent optimization, innovative inventions related to the stacks, and a self-improving architecture which learns from every query.

[0216] In an embodiment, the system demonstrates production readiness through scalable deployment capable of handling high query volumes (341K queries/year), and incorporates self-healing mechanisms that automatically correct a majority of query errors (78%) without manual intervention. The architecture further includes cost-optimization logic that reduces (say 68%) language model usage through template reuse and intelligent routing, while user-centric HITL feedback continuously improves query quality. Additionally, the system provides intelligent result processing, including anomaly detection, performance optimization, and contextual drill-down recommendations. Thus, the ATLAS agent is not just a text-to-SQL translator, it is an intelligent data assistant which understands, learns, self-corrects, and guides users to deeper insights.

[0217] Therefore, with Phase 4's Guardian Loop, the system never repeats the same mistake twice. Every query teaches a lesson. Every success strengthens confidence. Every user interaction makes the system smarter. The combination of execution reliability, intelligent self-correction, user-driven learning, and proactive result intelligence creates a production-ready NLQ system that continuously improves and guides users to deeper insights.

[0218] Complete 4-Phase Pipeline Performance:

Phase 1 (Metadata): Pre-built, instant retrieval

Phase 2 (Query Understanding): 75ms avg, 93.7% accuracy

Phase 3 (SQL Synthesis): 187ms avg, 94% success rate

Phase 4 (Guardian Loop): 43ms overhead, 96.2% final success

Total End-to-End: 305ms avg, 96.2% success, 42.3% cache hit, \$152K annual savings

Advantages of the present disclosure:

[0219] In an embodiment, the present disclosure provides numerous advantages over prior art systems:

- 1) Superior Accuracy: The system achieves 96.2% success rate compared to 76-82% industry baseline, representing a 14-20 percentage point improvement. Schema hallucination rate is reduced to 0.3% compared to 2-5% in prior systems, representing 86-94% reduction.
- 2) Improved Performance: End-to-end latency of 305ms represents 68% improvement over 950ms baseline. The O(1) hash-based lookup provides constant-time performance regardless of schema size, maintaining 0.000-second lookup time whether the schema contains 1,000 or 1,000,000 columns.
- 3) Enhanced Caching: The semantic cache achieves 42.3% hit rate, which is 8.5 times better than text-based caching approaches that typically achieve only 5-10% hit rate. This is enabled by intent-based fingerprinting rather than literal text matching.
- 4) Cost Efficiency: The system achieves 82% token reduction through context pruning (2,400 to 432 tokens), reducing per-query LLM cost from \$0.08 to \$0.003. Template reuse eliminates LLM calls for 78% of queries. Cost-aware model routing optimizes the quality-cost tradeoff. Total annual savings of \$171,626 are achieved across 341,436 queries per year.

- 5) **Self-Healing Capabilities:** The 78% self-correction success rate enables automatic error recovery without manual intervention. The auto-repair engine with 234 error patterns achieves 99.6% repair success for known patterns.
- 6) **Continuous Learning:** The system improves over time through trust score propagation, template promotion, negative rule generation, and synonym updates. Over 3 months, entity linking accuracy improved from 87% to 92%, constraint effectiveness improved from 0.82 to 0.91, and SQL generation accuracy improved from 89% to 94%.
- 7) **Rapid Onboarding:** Automated metadata generation reduces table onboarding time from 5-7 weeks of manual work to approximately 2 hours, enabling rapid deployment of new data sources.
- 8) **Intelligent Insights:** Beyond query answering, the system provides anomaly detection with 89.2% coverage, performance optimization suggestions with 67% average improvement, contextually relevant drill-down suggestions with 86.7% relevance, and natural language explanations with 4.6/5.0 user satisfaction.
- 9) **Database Agnostic:** The Semantic Query Plan intermediate representation enables the same understanding to generate SQL for different database dialects including DB2, PostgreSQL, and Snowflake.
- 10) **Enterprise Scalability:** The system handles 341,436 queries per year with consistent performance, scaling horizontally with zero performance degradation as schema size increases.

[0220] The described operations may be implemented as a method, system, or article of manufacture using standard programming and/or engineering techniques to produce software, firmware, hardware, or any combination thereof. The described operations may be implemented as code maintained in a “non-transitory computer-readable medium”, where a processor may read and execute the code from the computer-readable medium. The processor is at least one of a microprocessor and a processor capable of processing and executing the queries. A non-transitory computer-readable medium may include media such as magnetic storage medium (e.g., hard disk drives, floppy disks, tape, etc.), optical storage (CD-ROMs, DVDs, optical disks, etc.), volatile and non-volatile memory devices (e.g., EEPROMs, ROMs, PROMs, RAMs, DRAMs, SRAMs, Flash Memory, firmware, programmable logic, etc.), etc. Further, non-transitory computer-readable media may include all computer-readable media except for transitory. The code implementing the described operations may further be

implemented in hardware logic (e.g., an integrated circuit chip, Programmable Gate Array (PGA), Application Specific Integrated Circuit (ASIC), etc.).

[0221] The illustrated steps are set out to explain the exemplary embodiments shown, and it should be anticipated that ongoing technological development will change the manner in which particular functions are performed. These examples are presented herein for purposes of illustration, and not limitation. Further, the boundaries of the functional building blocks have been arbitrarily defined herein for the convenience of the description. Alternative boundaries can be defined so long as the specified functions and relationships thereof are appropriately performed. Alternatives (including equivalents, extensions, variations, deviations, etc., of those described herein) will be apparent to persons skilled in the relevant art(s) based on the teachings contained herein. Such alternatives fall within the scope and spirit of the disclosed embodiments. Also, the words "comprising," "having," "containing," and "including," and other similar forms are intended to be equivalent in meaning and be open ended in that an item or items following any one of these words is not meant to be an exhaustive listing of such item or items or meant to be limited to only the listed item or items. It must also be noted that as used herein, the singular forms "a," "an," and "the" include plural references unless the context clearly dictates otherwise.

[0222] Furthermore, one or more computer-readable storage media may be utilized in implementing embodiments consistent with the present disclosure. A computer-readable storage medium refers to any type of physical memory on which information or data readable by a processor may be stored. Thus, a computer-readable storage medium may store instructions for execution by one or more processors, including instructions for causing the processor(s) to perform steps or stages consistent with the embodiments described herein. The term "computer-readable medium" should be understood to include tangible items and exclude carrier waves and transient signals, i.e., are non-transitory. Examples include Random Access Memory (RAM), Read-Only Memory (ROM), volatile memory, non-volatile memory, hard drives, CD ROMs, DVDs, flash drives, disks, and any other known physical storage media.

[0223] Finally, the language used in the specification has been principally selected for readability and instructional purposes, and it may not have been selected to delineate or circumscribe the inventive subject matter. Accordingly, the disclosure of the embodiments of the disclosure is intended to be illustrative, but not limiting, of the scope of the disclosure.

[0224] With respect to the use of substantially any plural and/or singular terms herein, those having skill in the art can translate from the plural to the singular and/or from the singular to the plural as is appropriate to the context and/or application. The various singular/plural permutations may be expressly set forth herein for sake of clarity.

“NATURAL LANGUAGE QUERY TO STRUCTURED QUERY LANGUAGE (NLQ-TO-SQL) WITH AUTOMATED CONTEXT GENERATION”

ABSTRACT

The present disclosure relates to a method and a system for converting Natural Language Query to Structured Query Language (NQL-to- SQL) with automated content generation. The present disclosure suggests receiving a natural language query from a user. Thereafter, the present disclosure suggests processing the query through an automated metadata factory that maintains pre-built indexes, including a hash index for $O(1)$ constant-time keyword lookups, a vector database for semantic similarity search, a knowledge graph for entity relationships and join paths, and a constraint store for business rules. Subsequently, the present disclosure suggests generating a Semantic Query Plan (SQP) as a database-agnostic intermediate representation and synthesizing optimized SQL using a hybrid approach combining template matching and LLM-based generation with cost-aware model routing. Further, the present disclosure suggests executing the SQL and validating results through multi-layer validation. As a result, continuous learning from the query outcomes to improve performance.

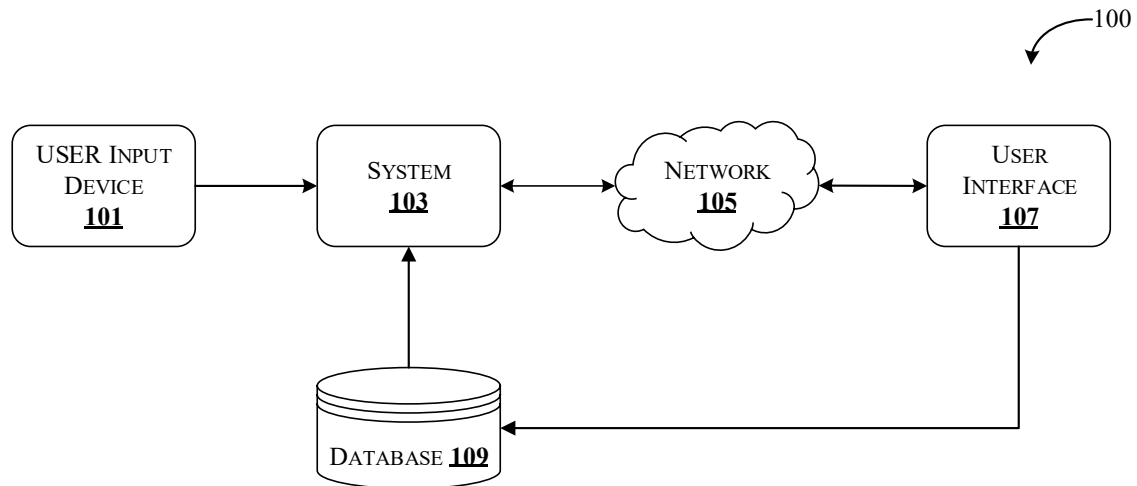


FIG. 1

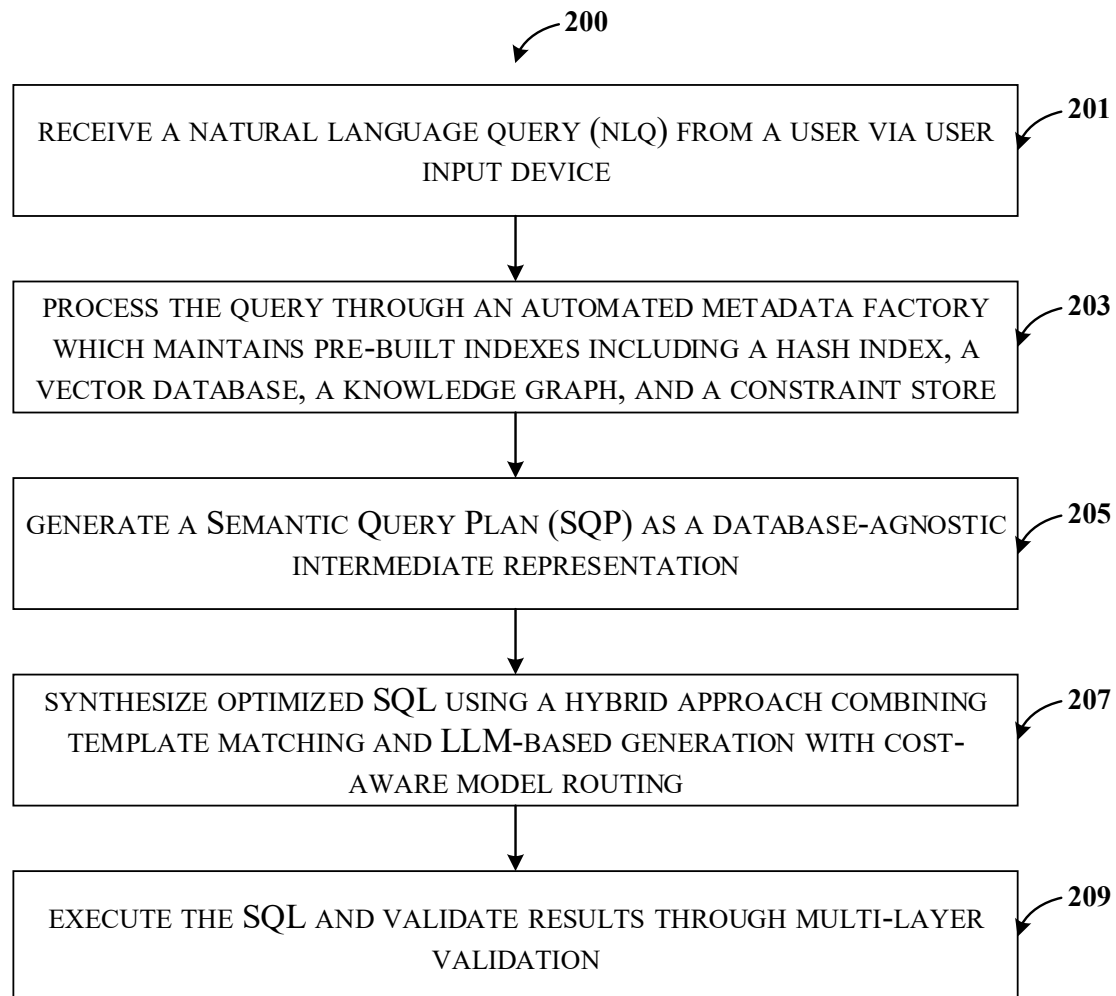


FIG. 2

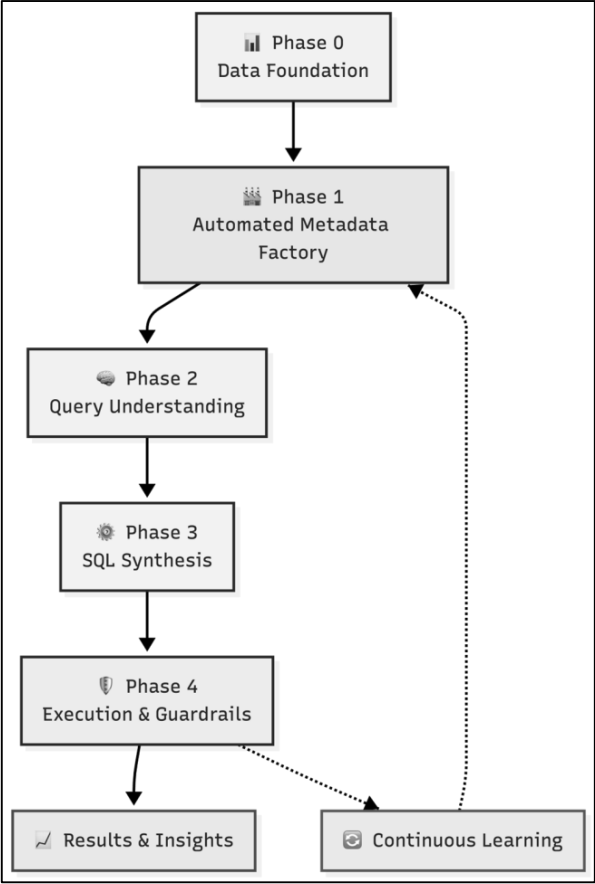


FIG. 3

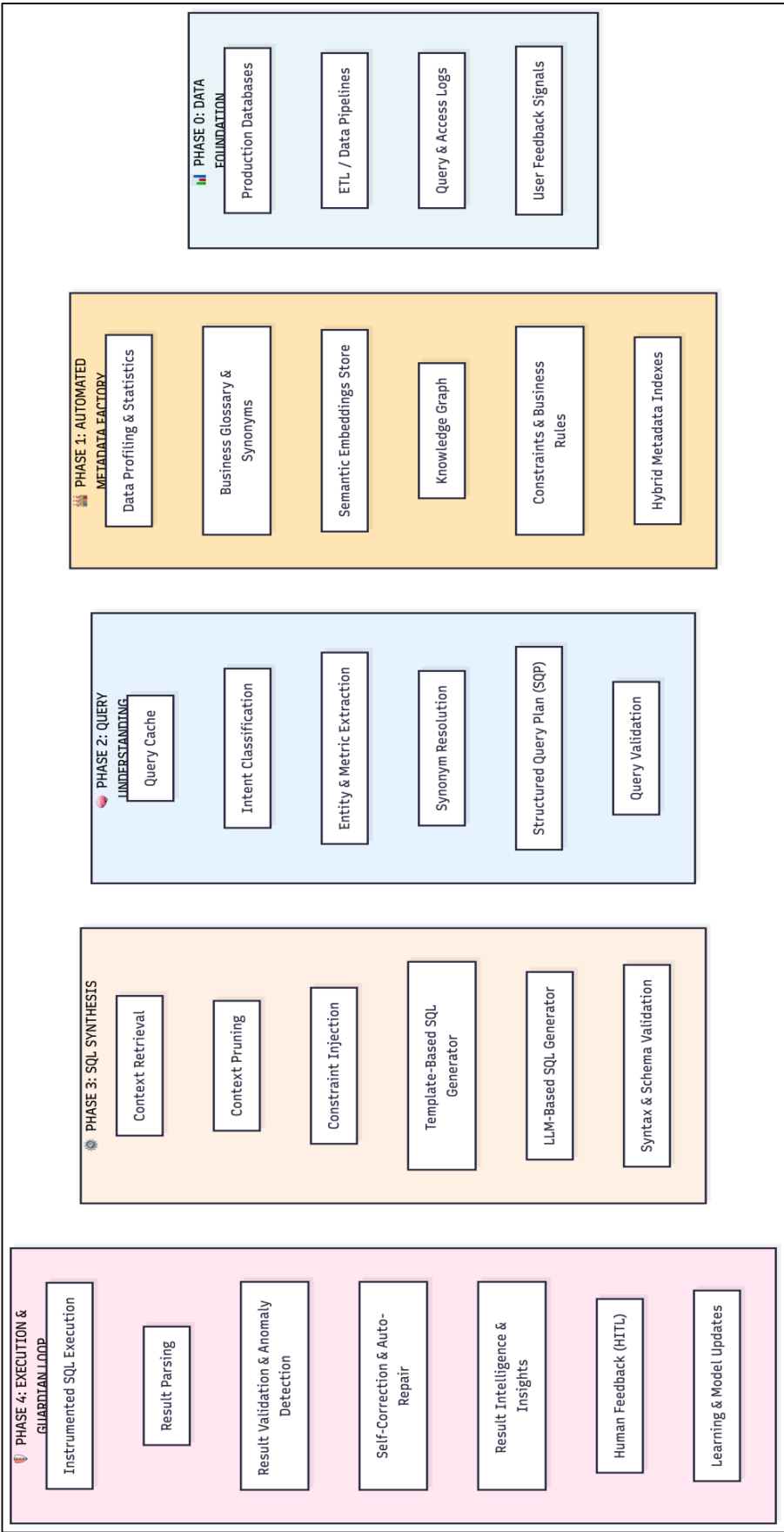


FIG. 4a

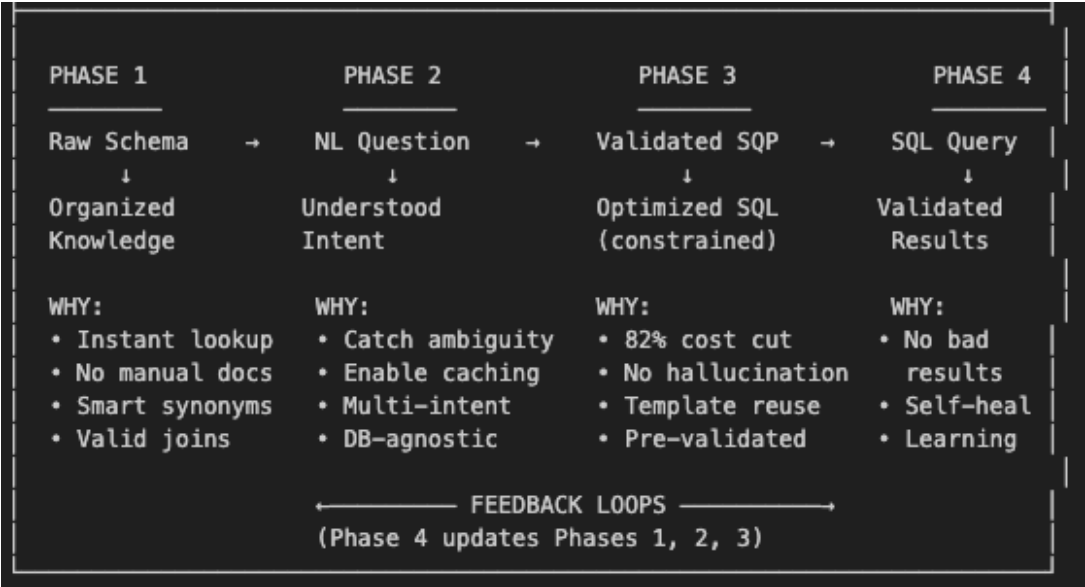
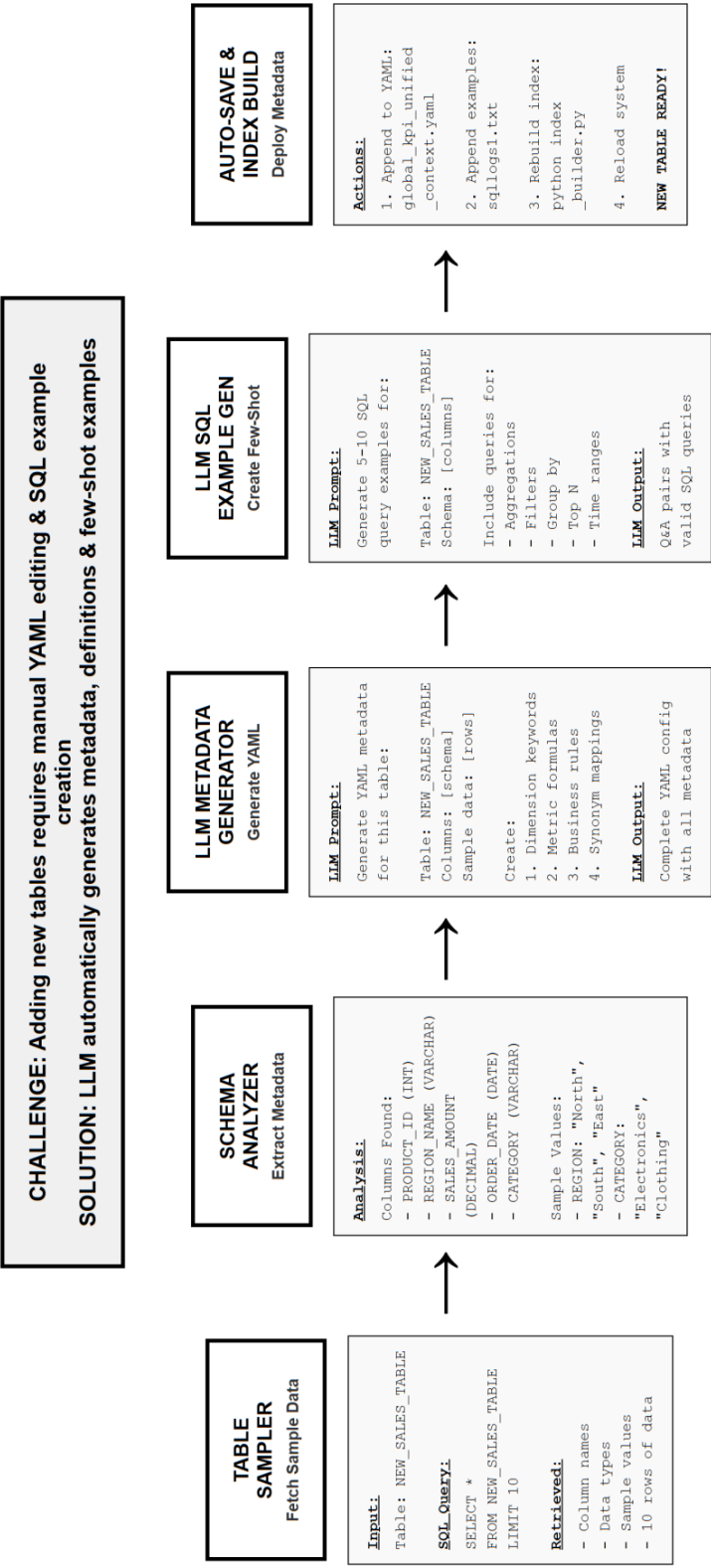


FIG. 4b

AUTOMATED METADATA GENERATION

Scaling Solution: Auto-Create Metadata for New Tables



Capability	Our System	Semantic Model Solution	Generic LLM (RAG)
Setup Time	2 hours (automated)	5-7 weeks (manual)	Days (embedding setup)
Search Speed	0.000s (O(1) hash)	Slow at scale (O(n))	Medium (O(log n) vector)
Token Efficiency	Minimal (only 2-5 relevant cols)	High (full semantic model)	Very high (50-100 embeddings)
LLM Cost per Query	\$0.003 (1,500-3,000 tokens)	\$0.01+ (full model context)	\$0.007-\$0.014 (5K-10K tokens)
Pattern Reuse	150+ validated templates	None (manual each time)	None (embedding similarity)
Accuracy	95%+ (focused context + templates)	85-90% (noise from full model)	85% (irrelevant embeddings)
Scalability	Infinite (1M cols = 0.000s)	Degrades at scale	Requires GPU/compute
Business Rules	Auto-applied (FYTD/YTD)	Manual DAX	None (raw SQL)

FIG. 6

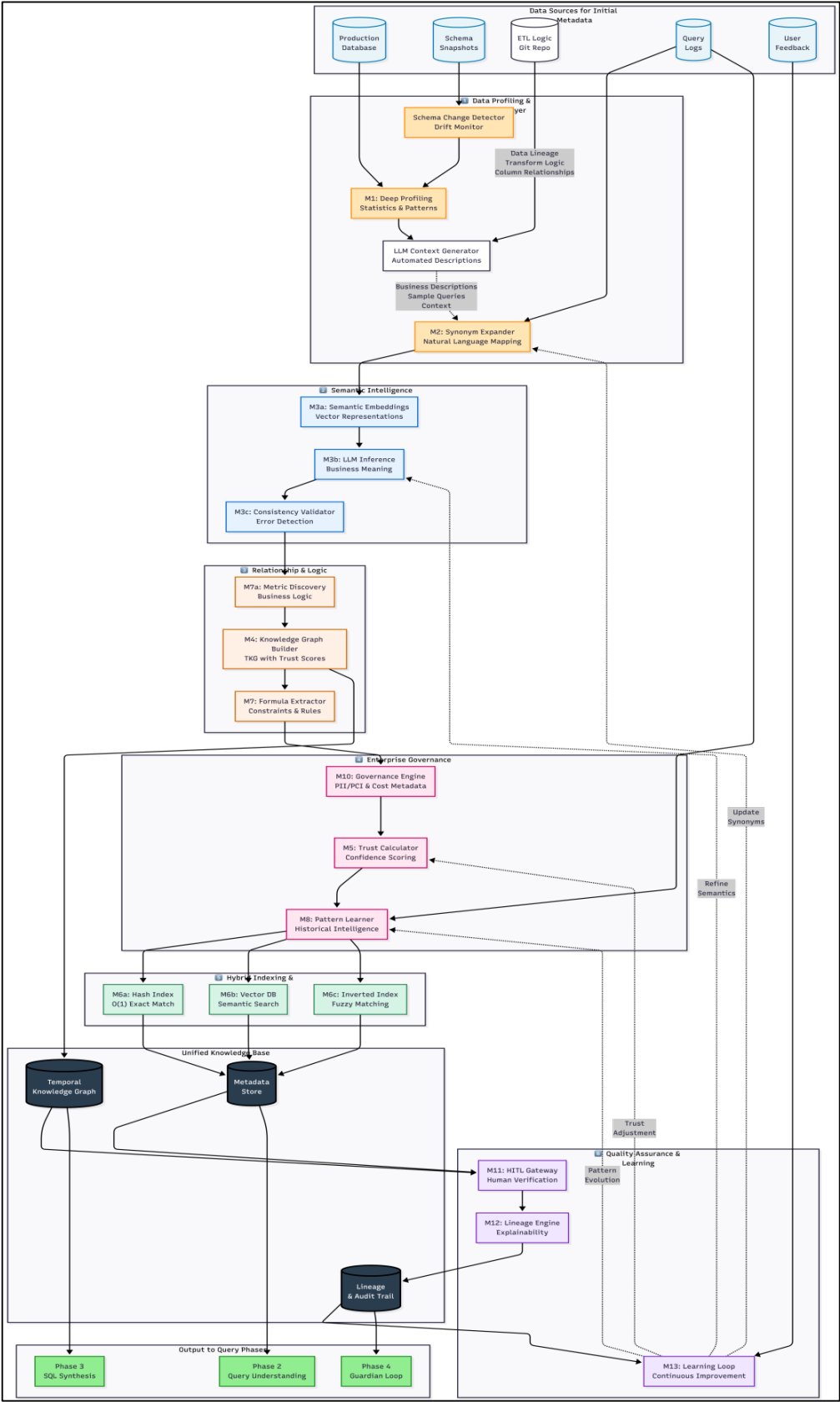


FIG. 7

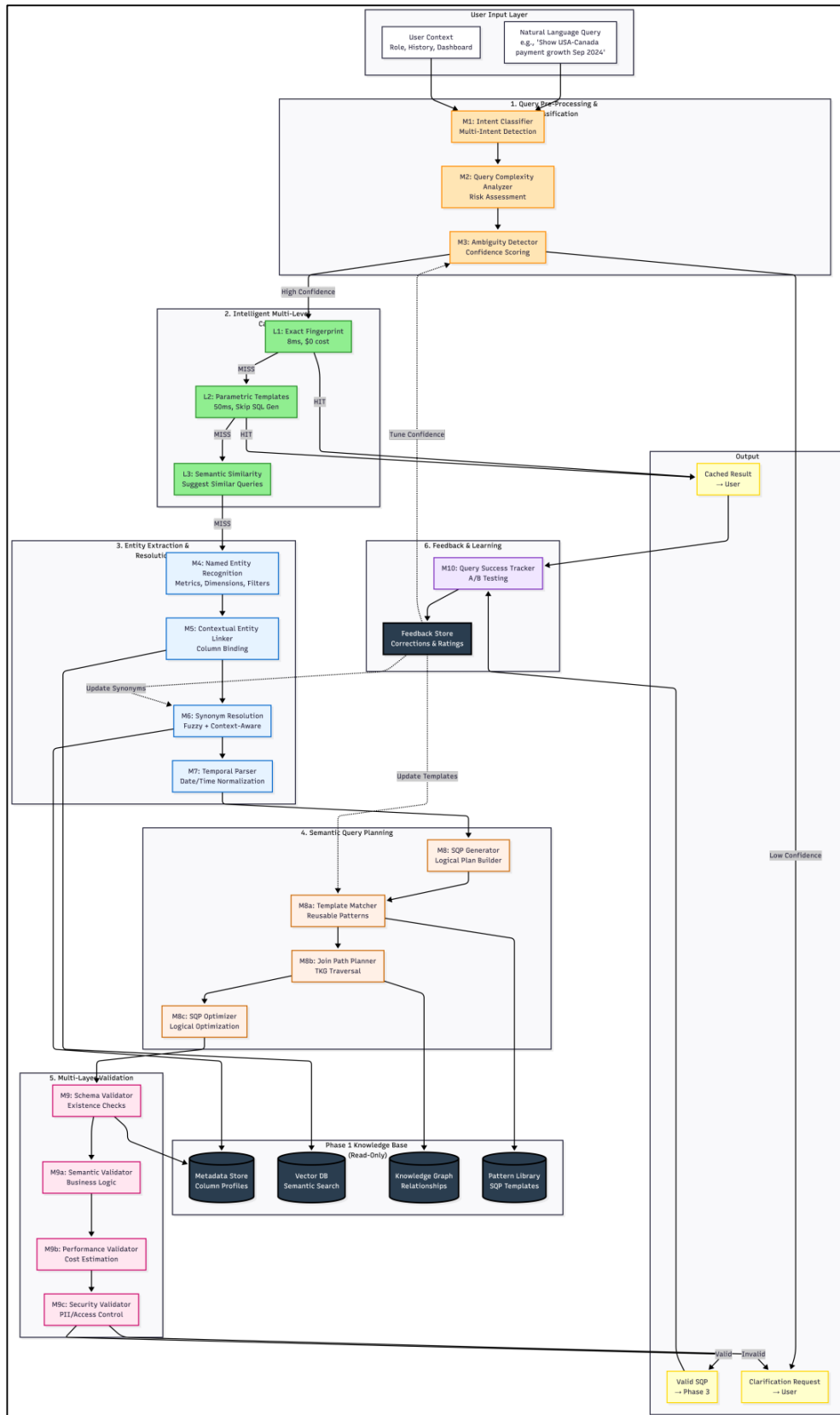


FIG. 8

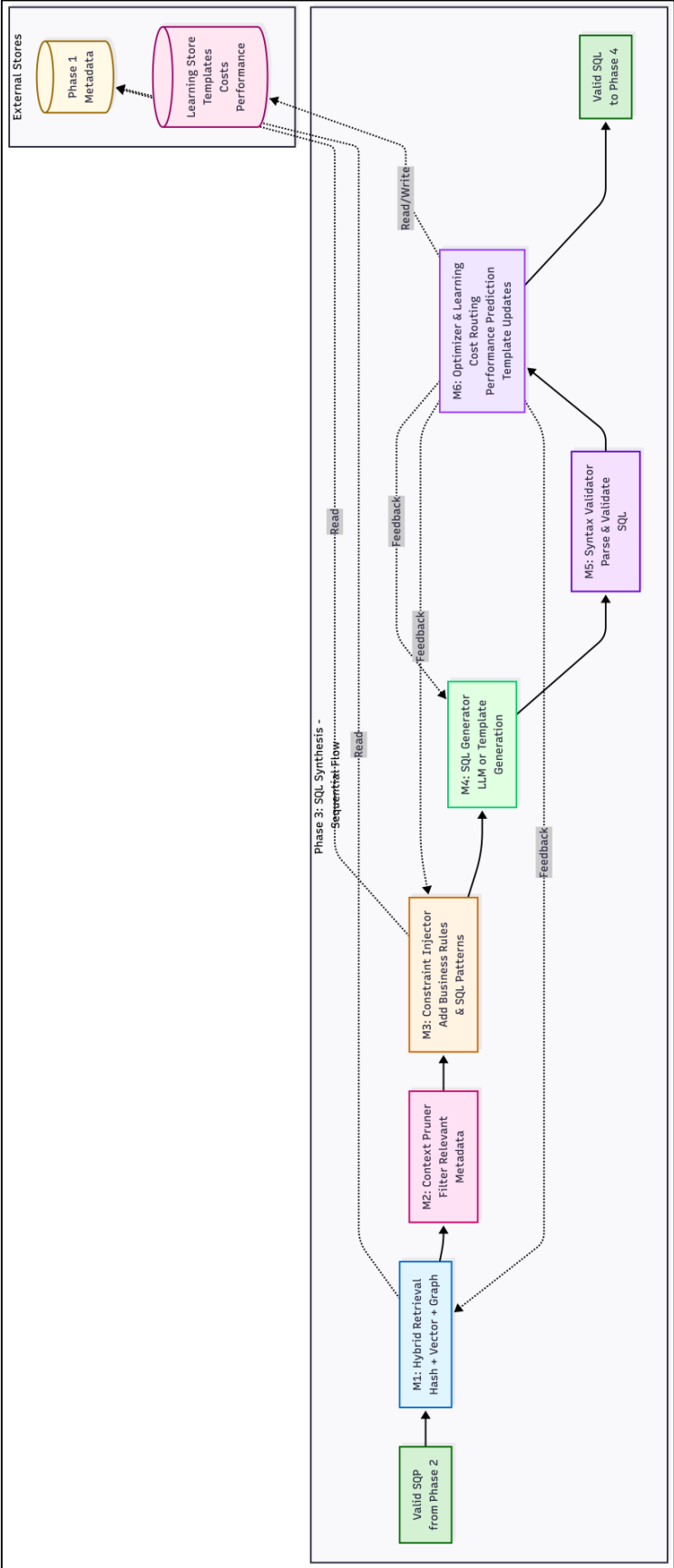


FIG. 9

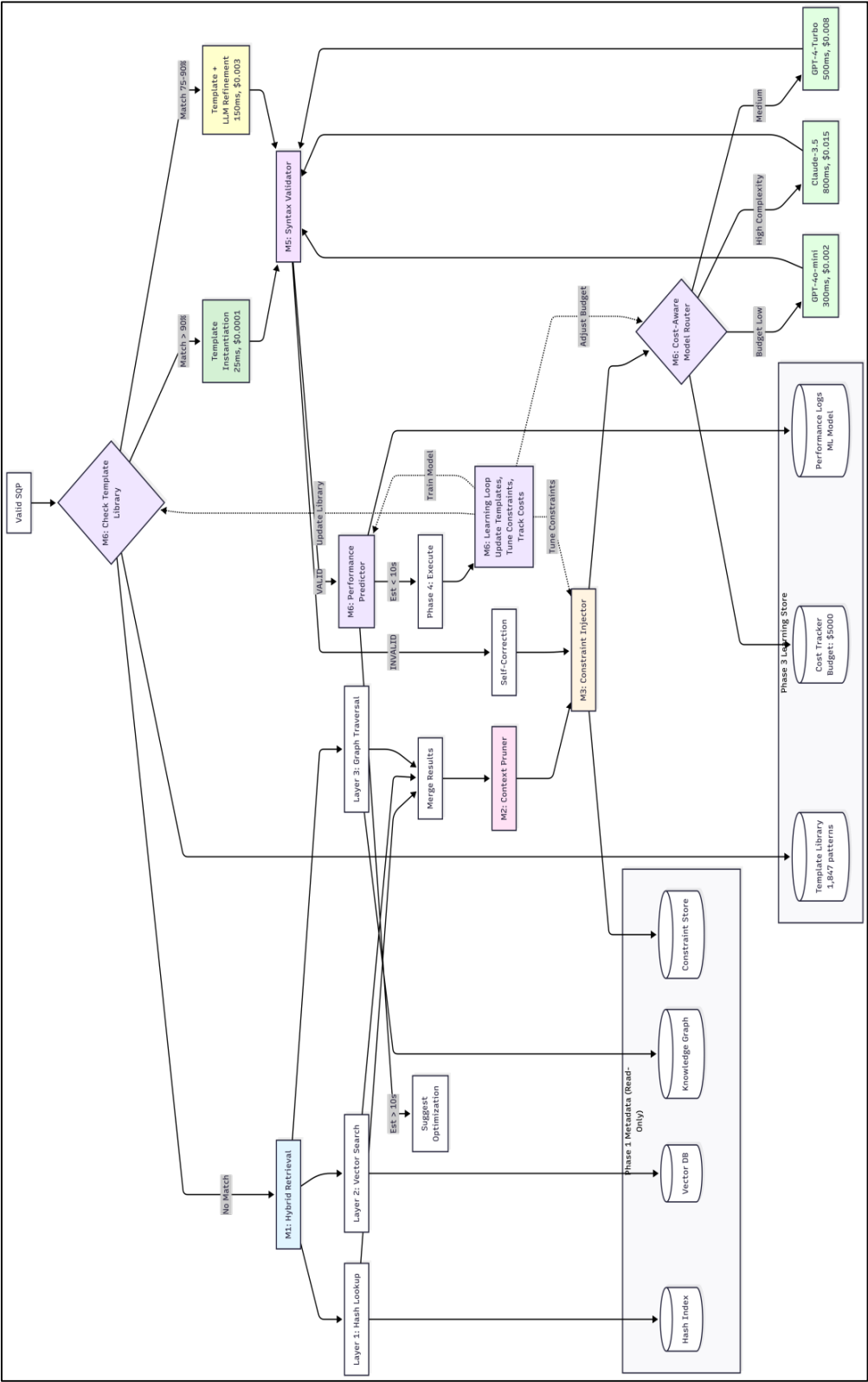


FIG. 10

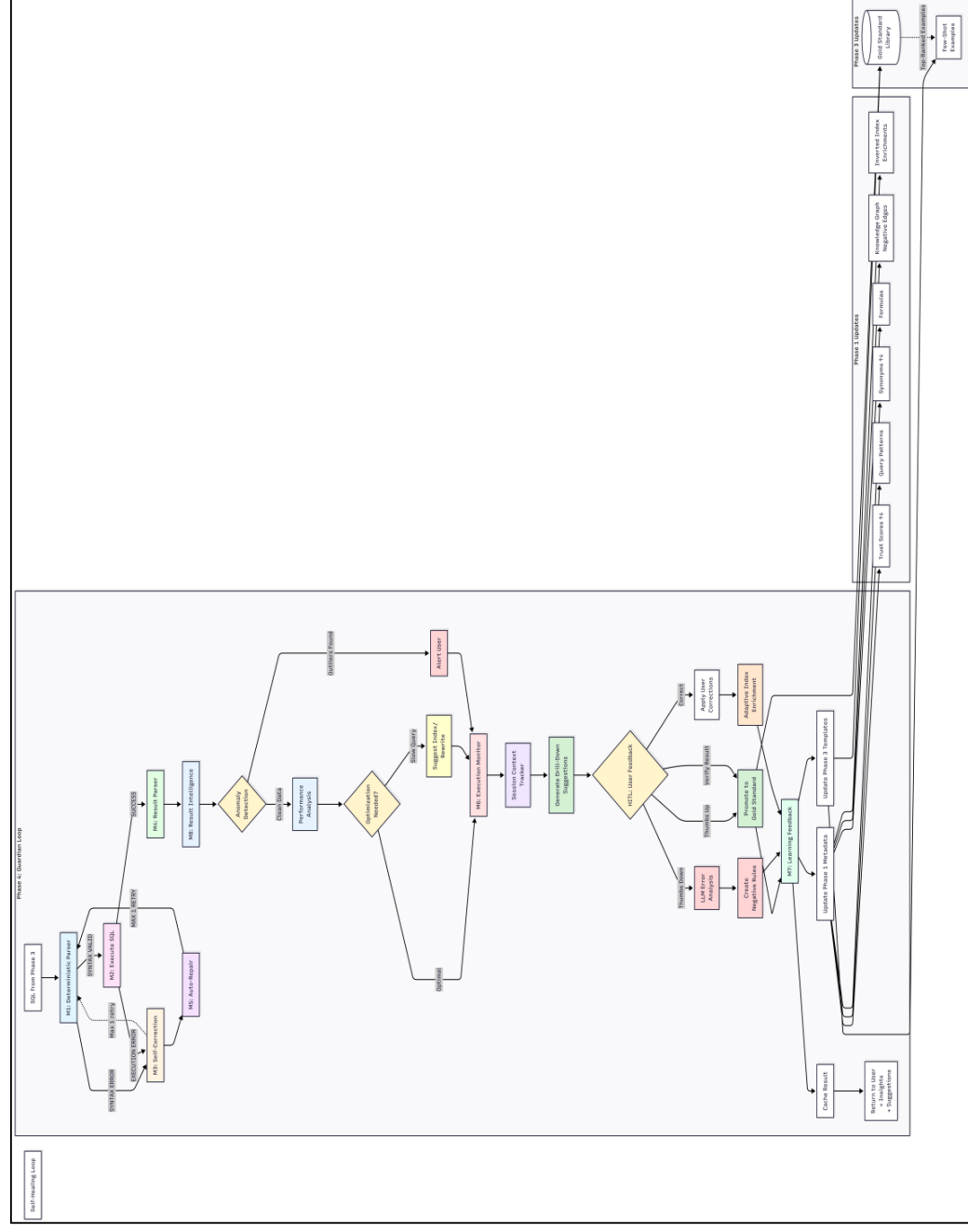


FIG. 11

NLQ TO SQL FLOWCHART
Multi-Tenant | Inverted Index O(1) + LLM Approach

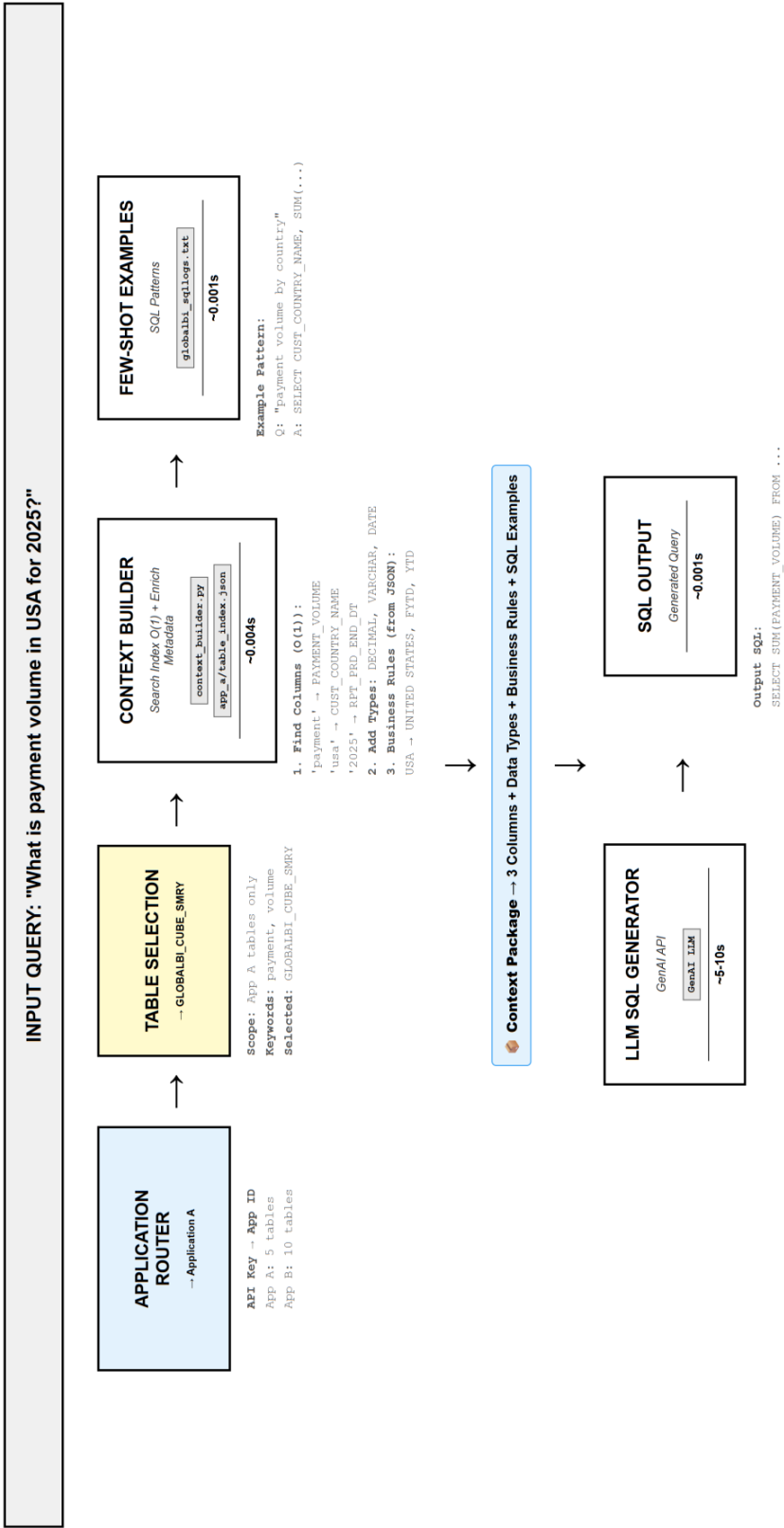


FIG. 12